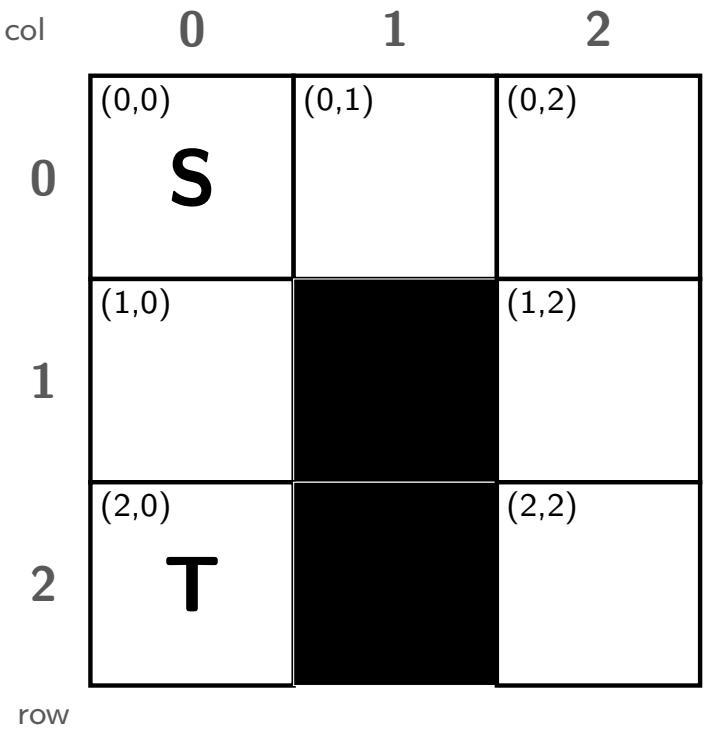
Slip Question 3: DFS, Slowly

CSCI 235

Slip Question 3: The Setup



- current call
- visited, call still active (?)
- dead end: returned empty-handed
- on the final route

Each call marks its cell visited and tries its neighbors in the order left, right, up, down. Black cells are walls.

The Function We Are Tracing

Every box on the call stack is one call of this recursive helper. `visited` and `pathStack` are passed **by reference**, so all calls share them.

```
void helper(const std::vector<std::vector<int>>& maze,
            const Coord& curr, const Coord& target,
            std::vector<std::vector<bool>>& visited,
            std::stack<Coord>& pathStack)
```

1. Mark `curr` visited. (Never unmark it.)
2. If `curr` is the target: push `curr` onto `pathStack` and return.
3. Try the neighbors in the order left, right, up, down.
4. If a neighbor is open and unvisited: call `helper` on it.
5. After that call: if `pathStack` is **not empty**, the target was found through this neighbor. Push `curr` and return.
6. Otherwise the call returned empty-handed: go on to the next neighbor. Falling off the end returns to our caller (that is the backtrack).

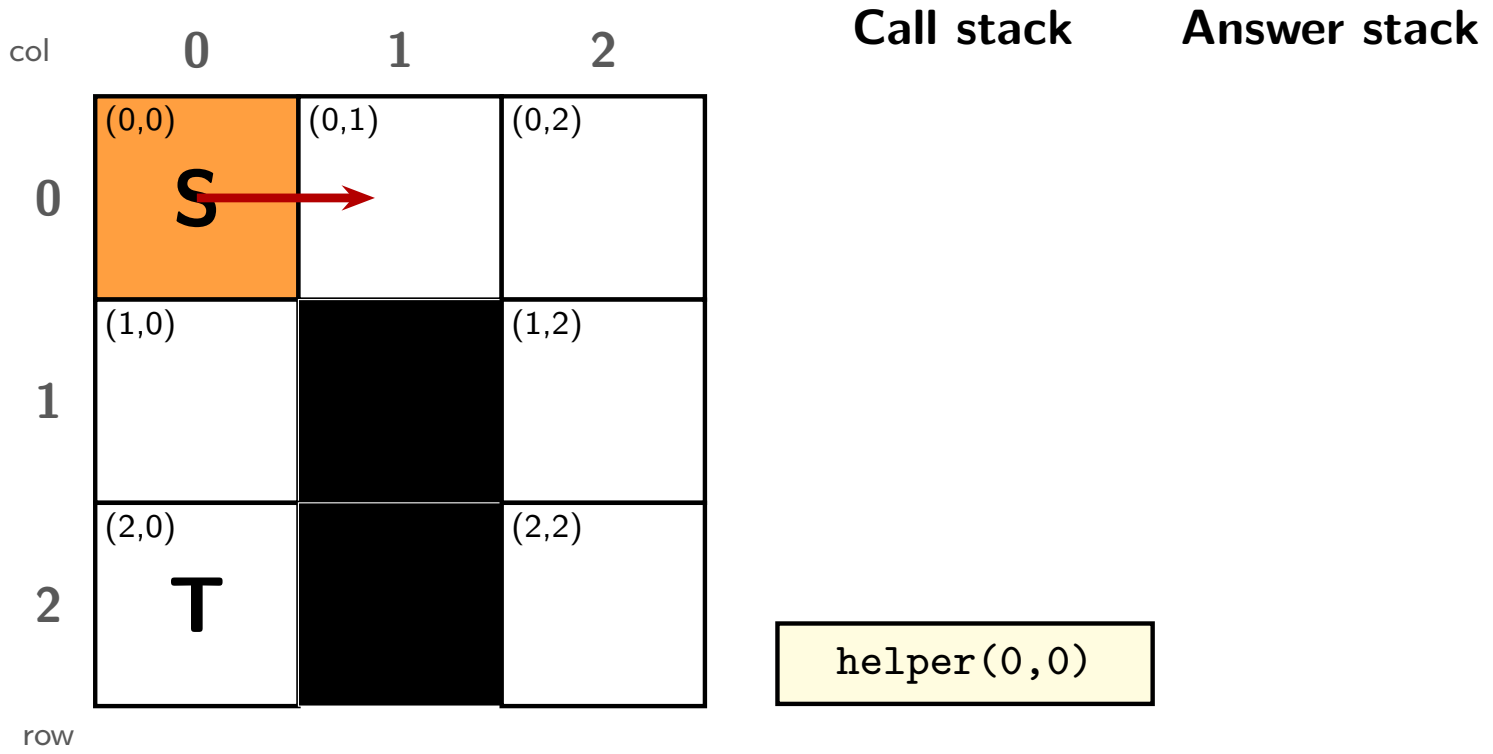
On the next slides, `helper(r,c)` means a call whose `curr` is (r, c). The trace has no parent table: the active calls already remember the way back.

Two Things to Notice About This Function

1. `pathStack` **starts empty, and that matters.**
   - Before the first call, `pathStack` is empty. It stays empty while DFS wanders and hits dead ends.
   - When the target is reached, the target is pushed. As the calls return, each cell on the successful route pushes itself.
   - So a nonempty `pathStack` does two jobs: it **stores the route**, and it **signals "found it"**. That is why step 5 only needs to ask `pathStack.empty()`.

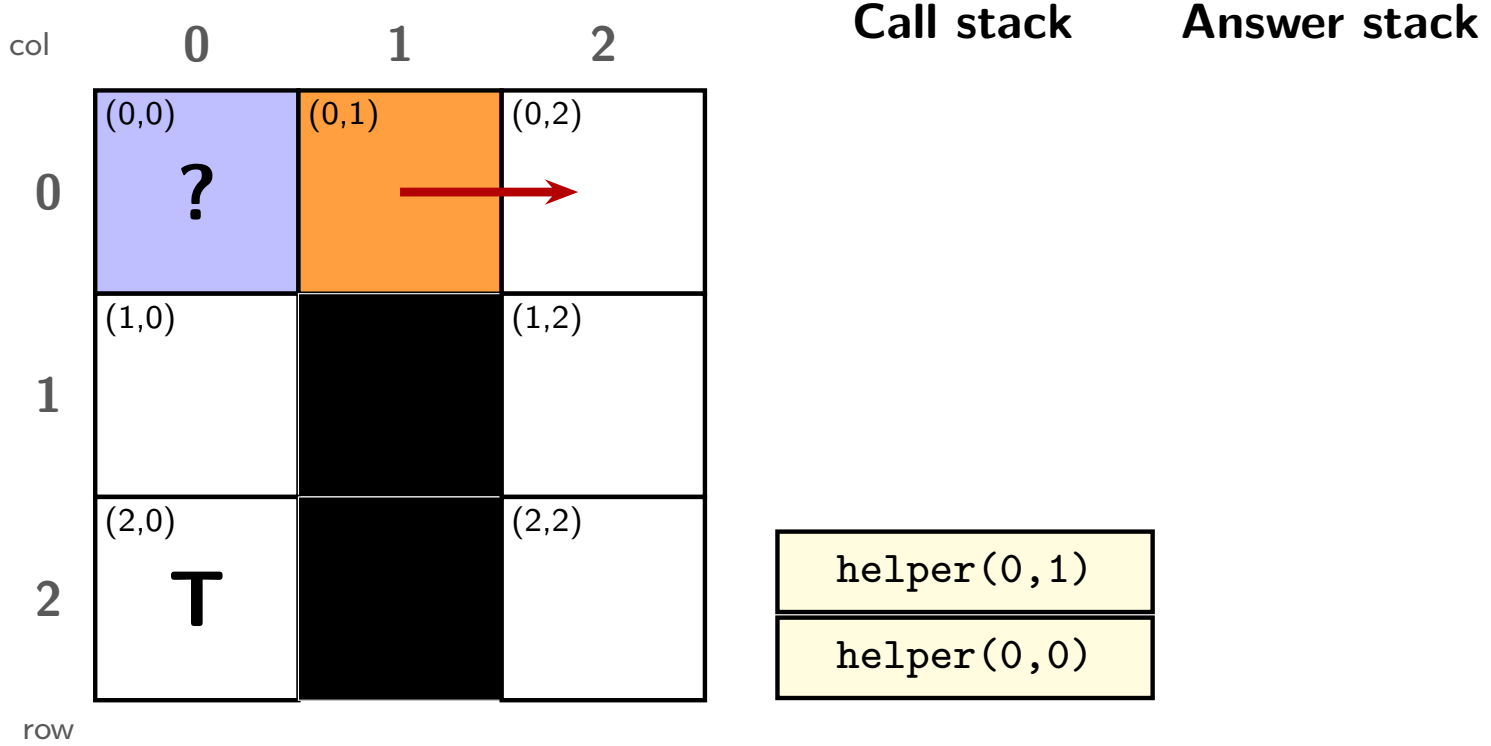
2. **We never unmark a visited cell.**
   - Think of chalk marks in a maze: once you have explored everything past a cell and found no target, coming back to it by another corridor cannot reveal anything new.
   - DFS here looks for **one** route, not every route. (A search that lists all possibilities, such as the Make 24 search, does undo its choices. This one does not.)

Slip Question 3, Step 1 of 18



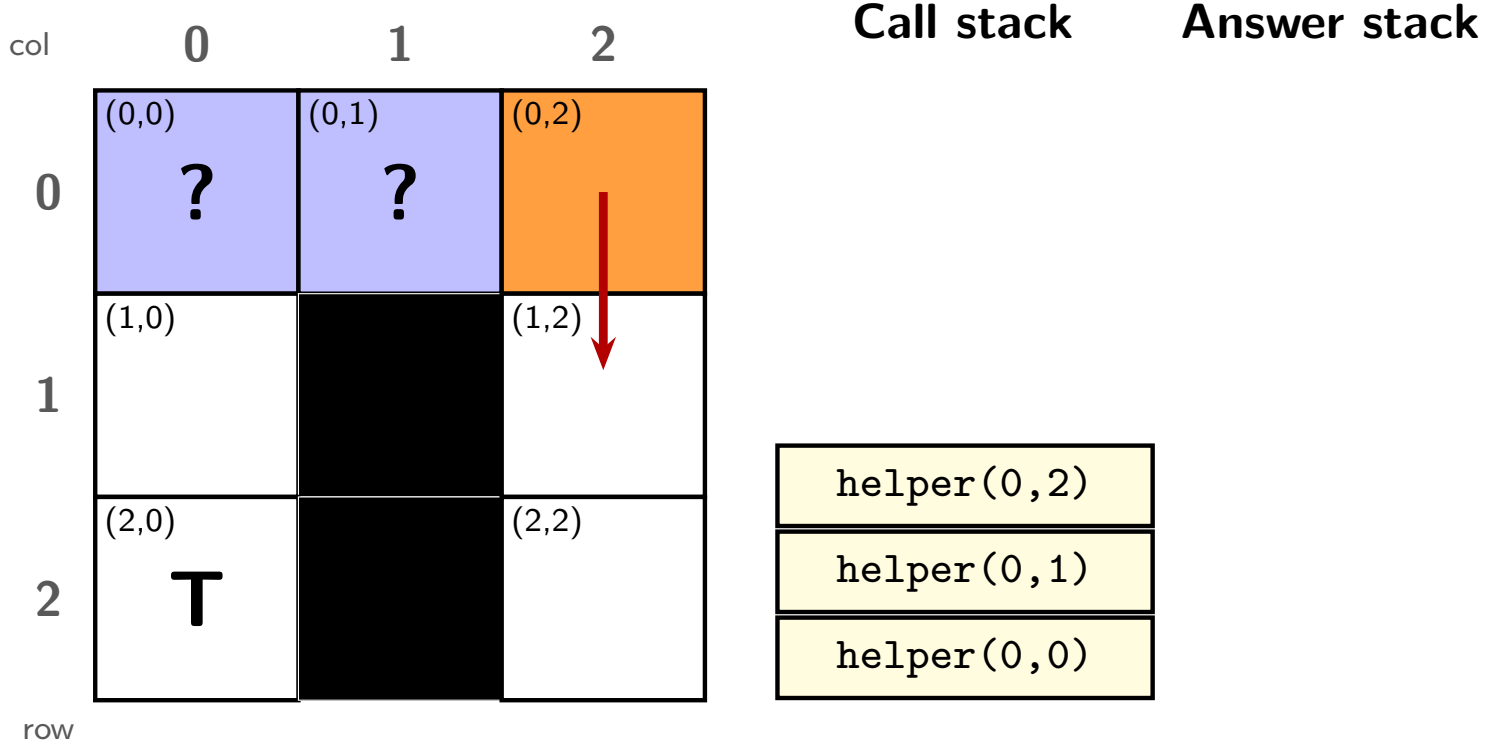
Start: `helper(0,0)`. Mark $(0, 0)$ visited. Left is outside the grid, so try **right**: $(0, 1)$ is open and unvisited. **Call** `helper(0,1)`.

Slip Question 3, Step 2 of 18



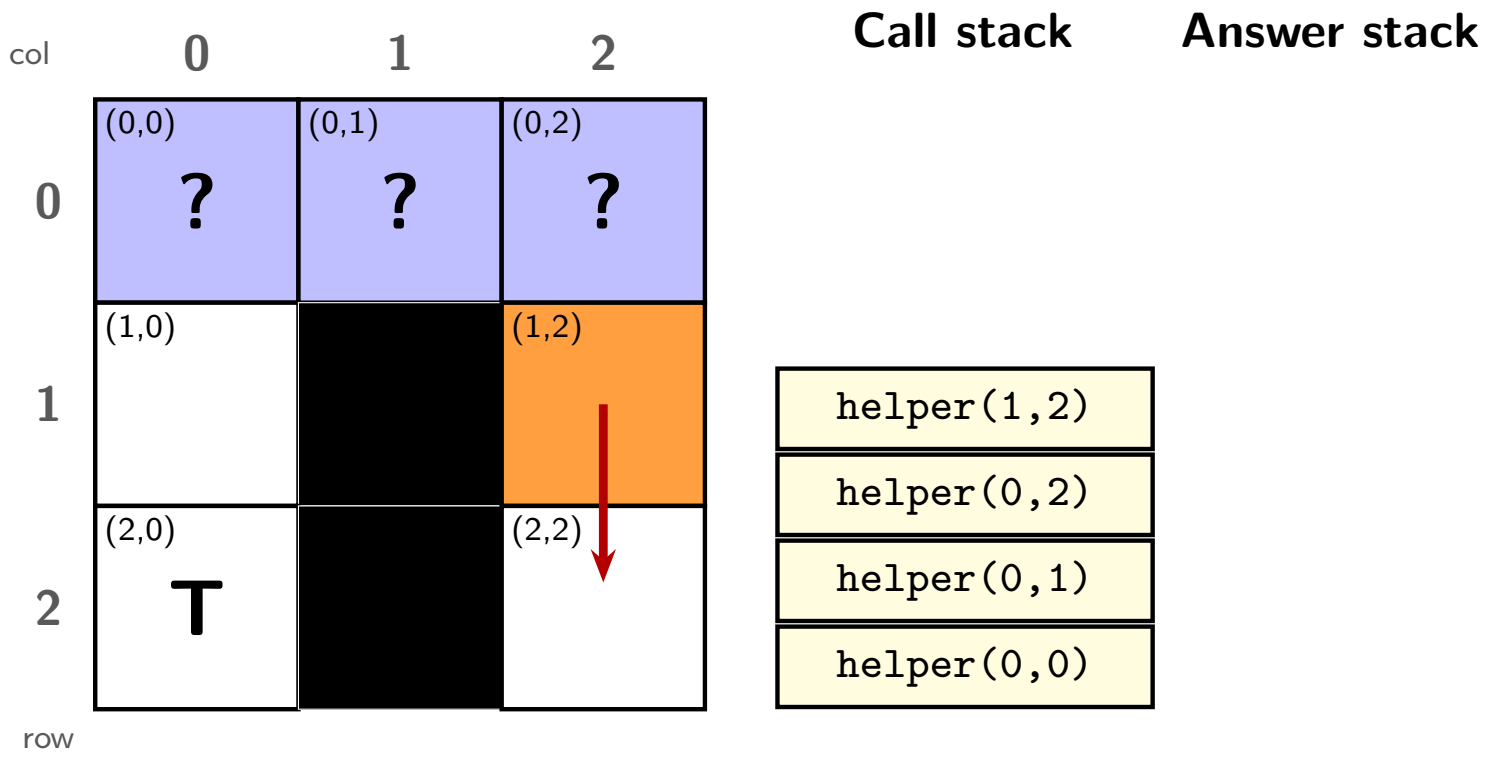
`helper(0,1)`: left $(0,0)$ is visited, below is a wall. The only choice is right, $(0,2)$. **Call** `helper(0,2)`. The ? on $(0,0)$ means: still waiting for an answer.

Slip Question 3, Step 3 of 18



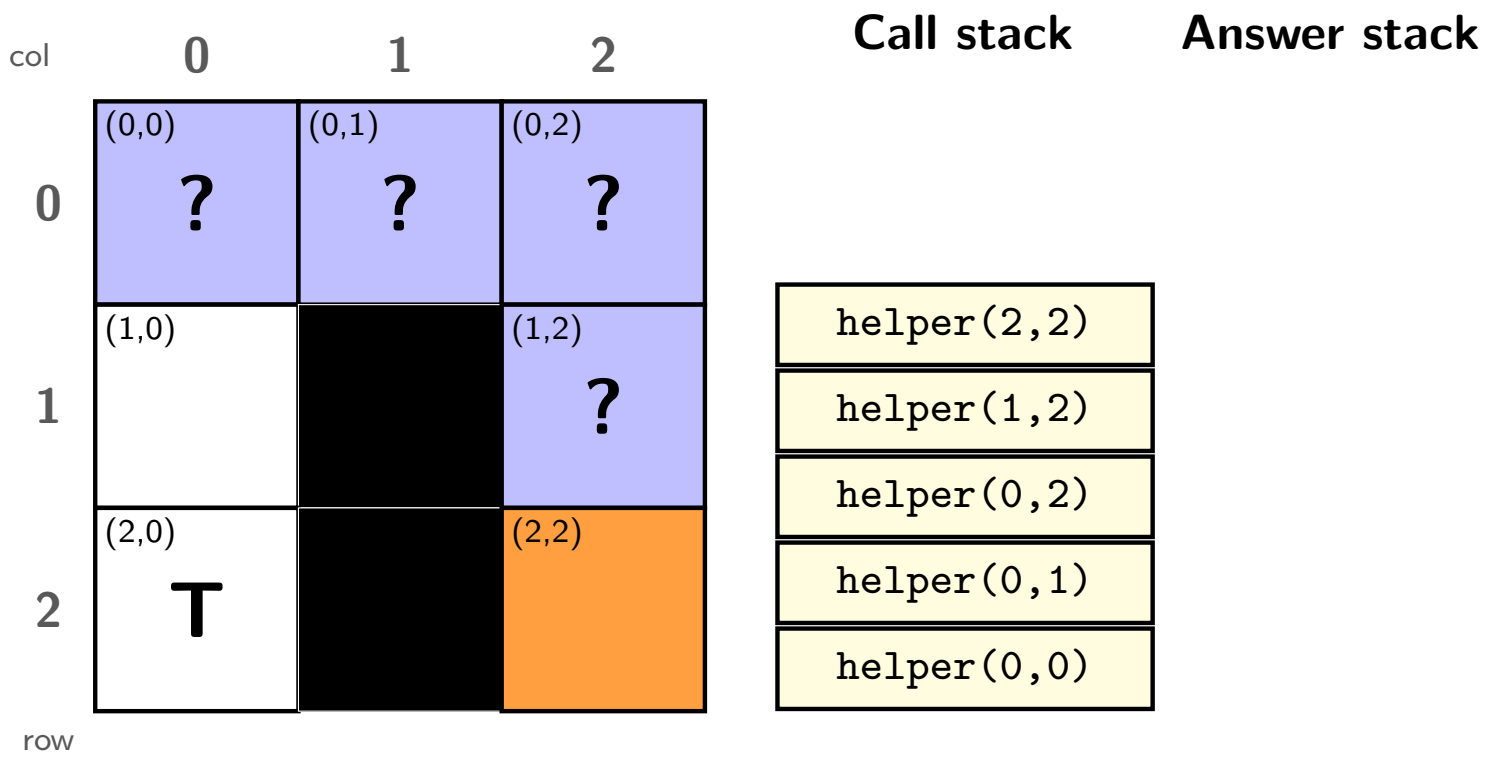
`helper(0,2)`: left is visited, right and up are outside the grid. The only choice is down, $(1, 2)$. **Call** `helper(1,2)`.

Slip Question 3, Step 4 of 18



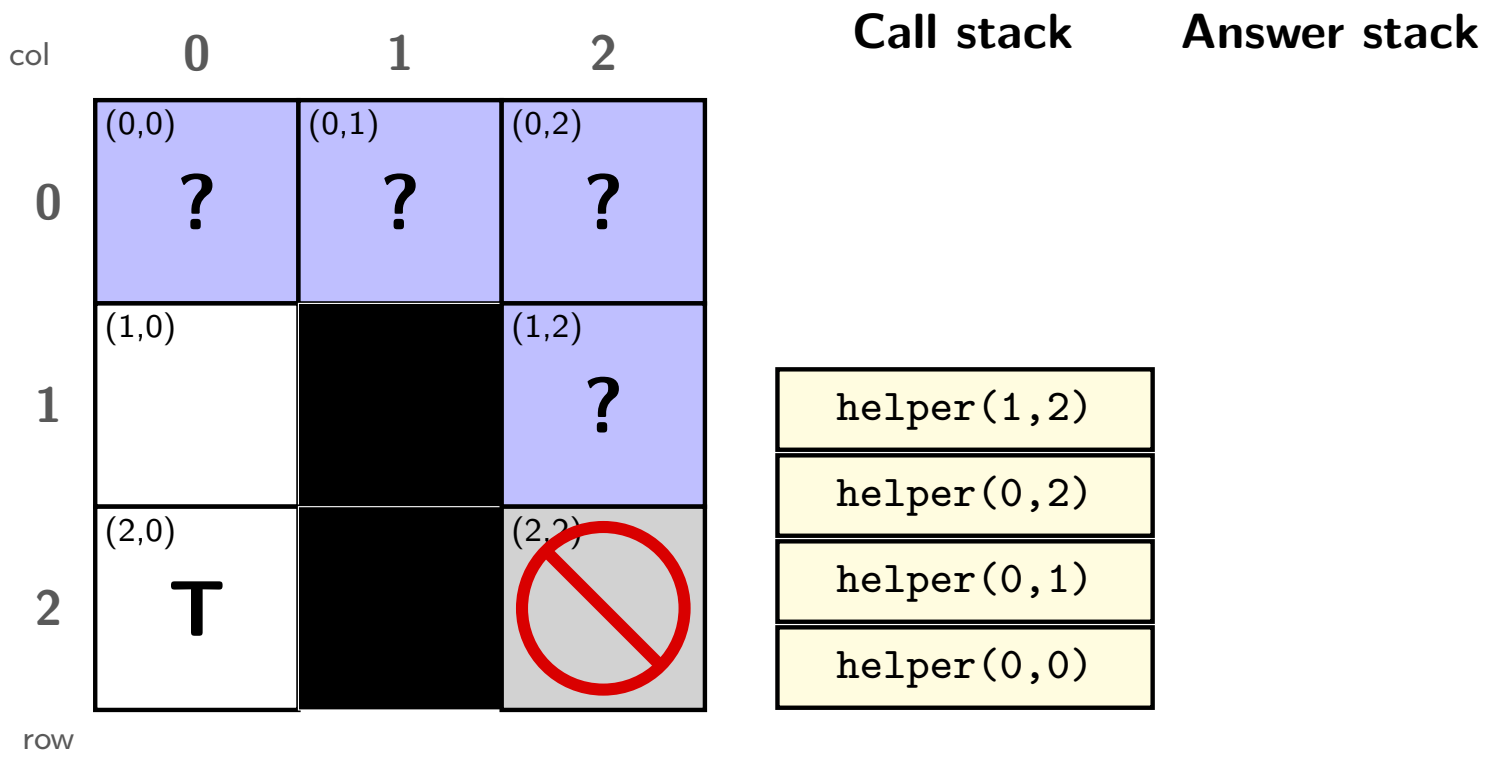
`helper(1,2)`: left is a wall, right is outside, up is visited. The only unvisited neighbor is down, $(2, 2)$. **Call** `helper(2,2)`.

Slip Question 3, Step 5 of 18



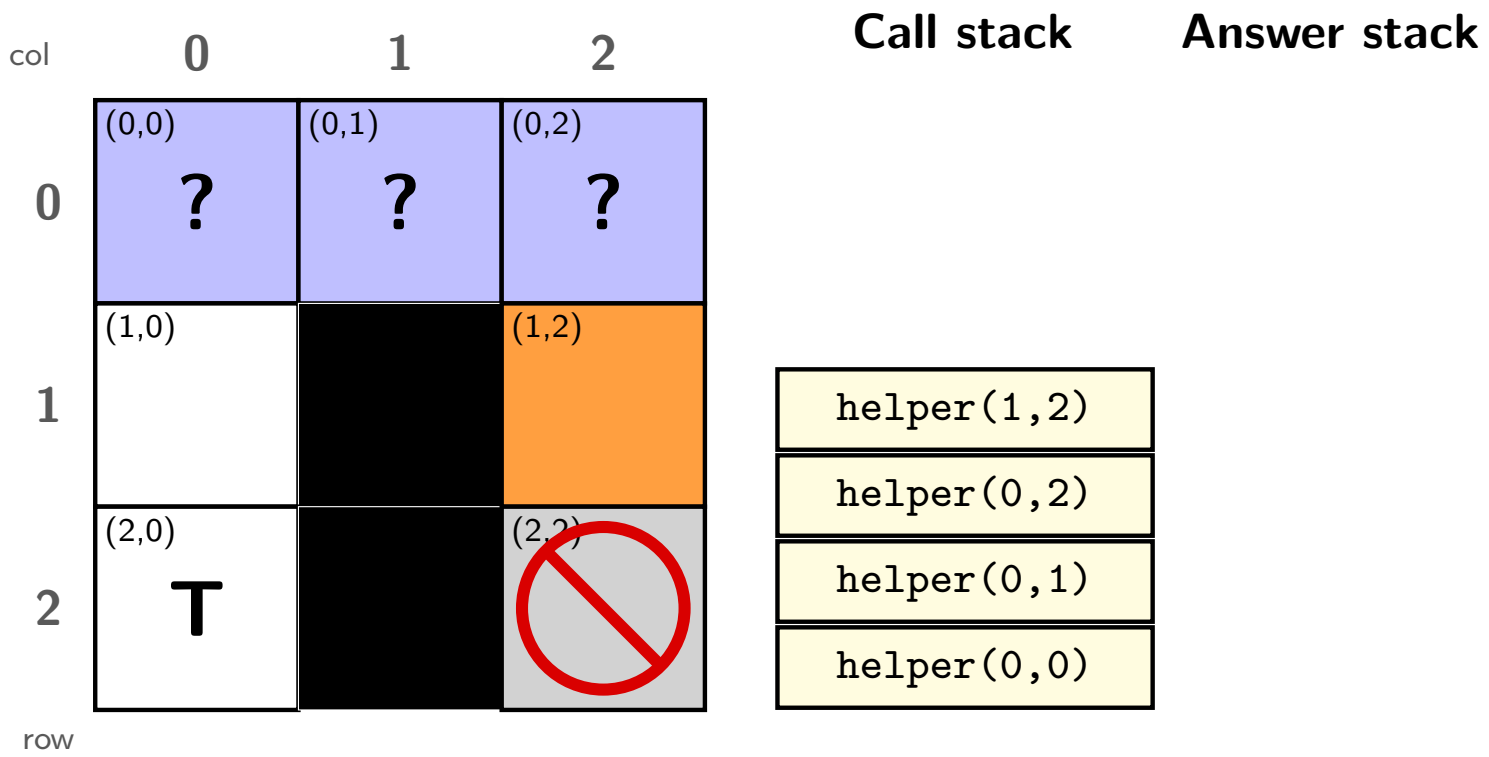
`helper(2,2)`: left is a wall, right and down are outside, up is already visited. No neighbor left to try: a **dead end**. It is not the target, so there is nothing to put on the answer stack.

Slip Question 3, Step 6 of 18



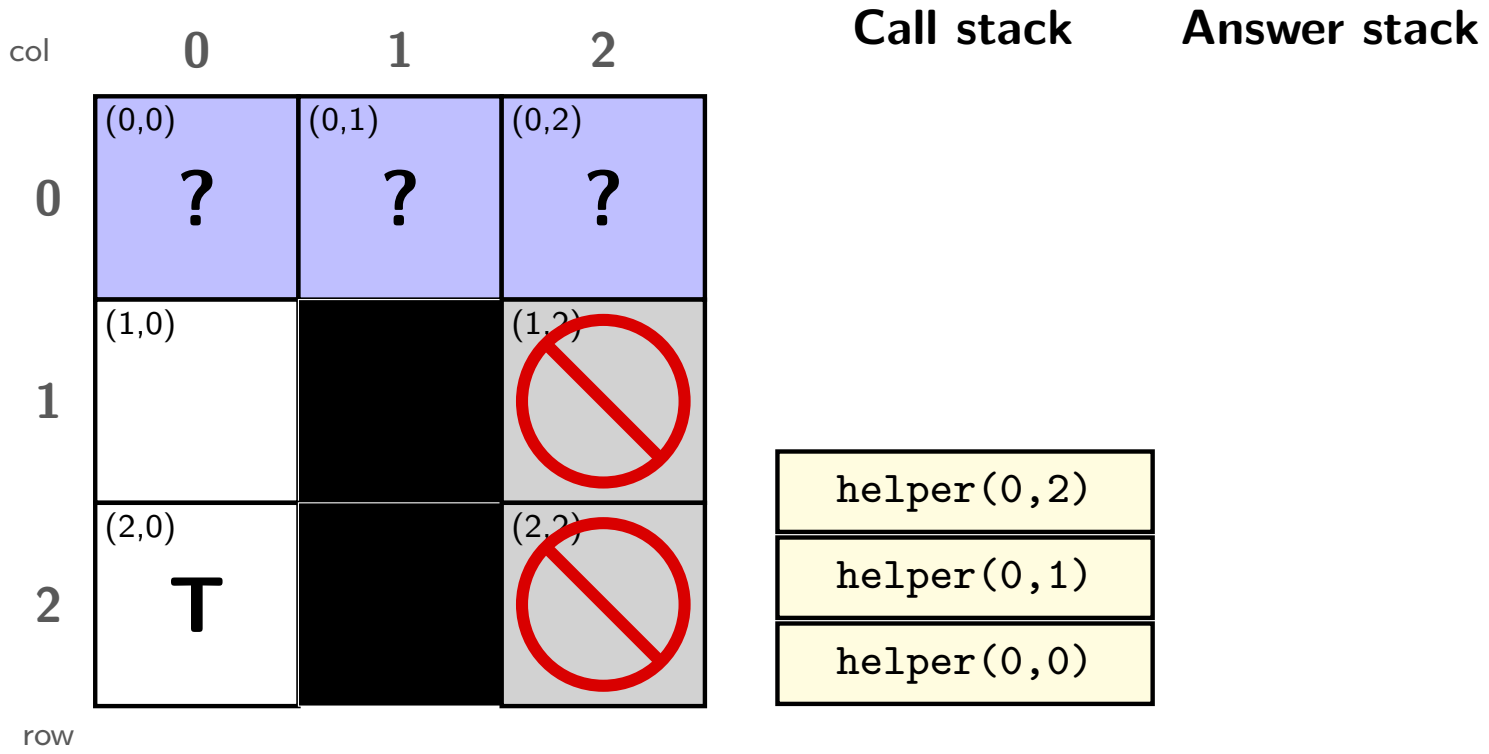
`helper(2,2)` **returns empty-handed**: it gets the stop sign, and its frame leaves the call stack. The answer stack is still empty. Now **backtrack** to the caller, $(1, 2)$.

Slip Question 3, Step 7 of 18



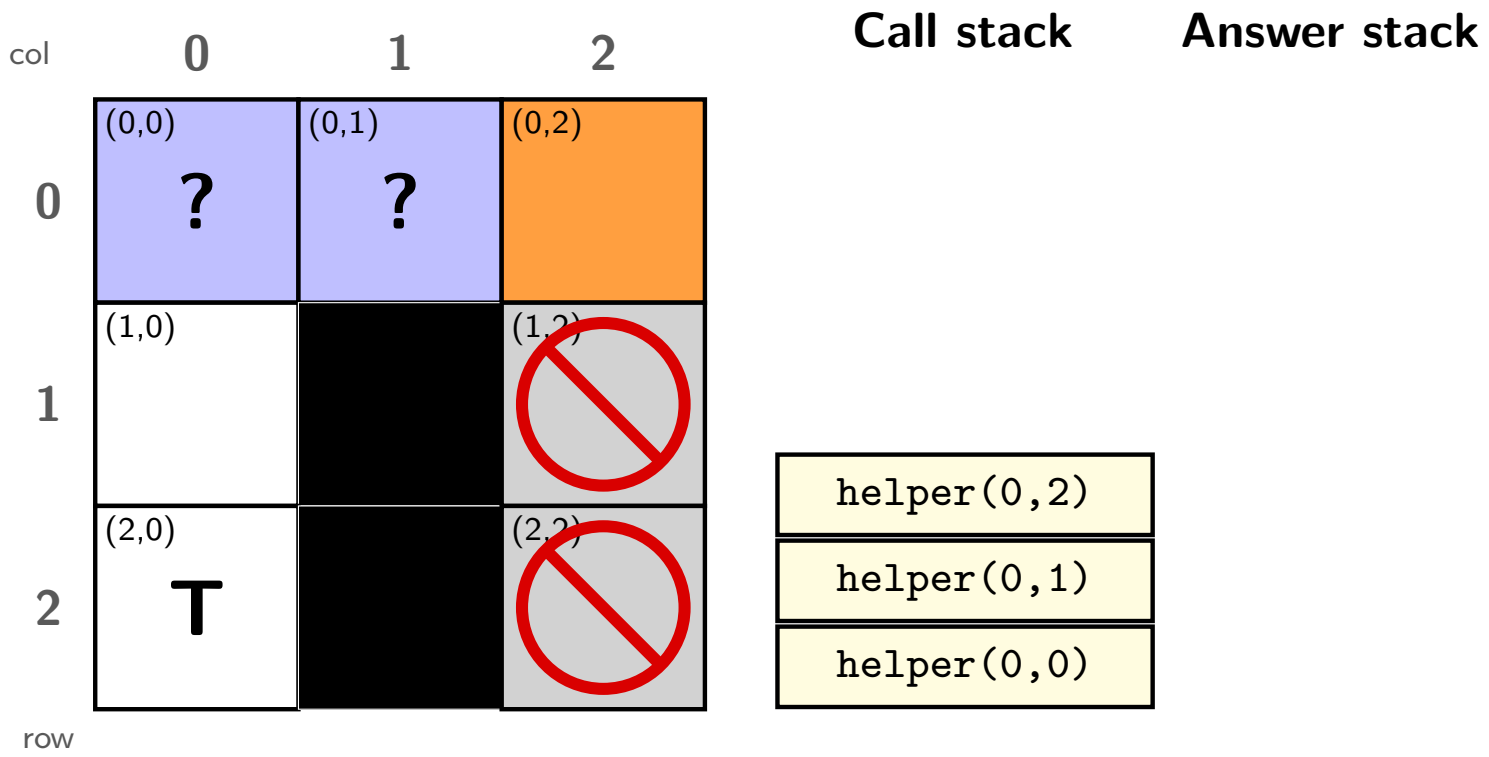
Back in `helper(1,2)` (its ? is about to be answered). The call came back with nothing, and $(1, 2)$ has no other unvisited open neighbor to try.

Slip Question 3, Step 8 of 18



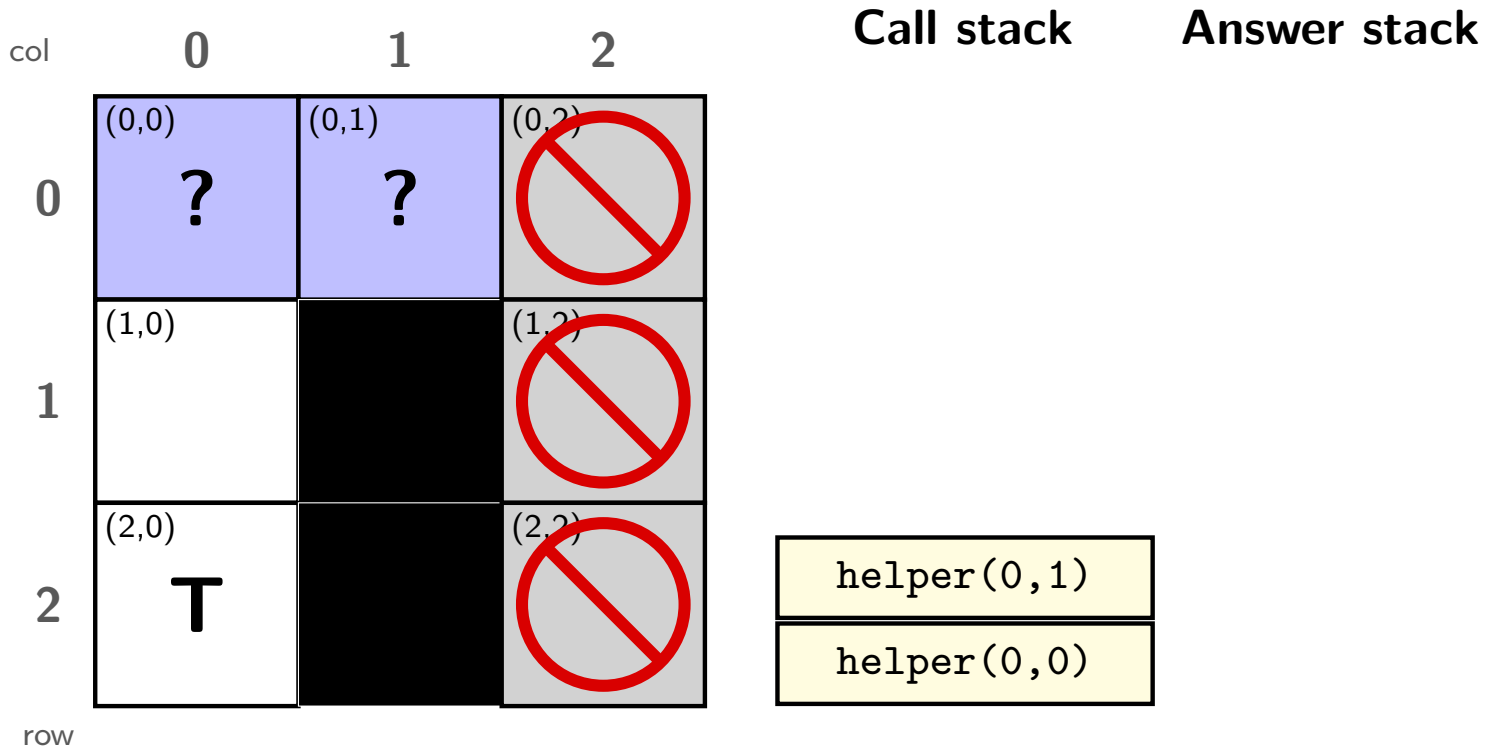
So $(1, 2)$ also **returns empty-handed**: ? becomes a stop sign. Backtrack to $(0, 2)$.

Slip Question 3, Step 9 of 18



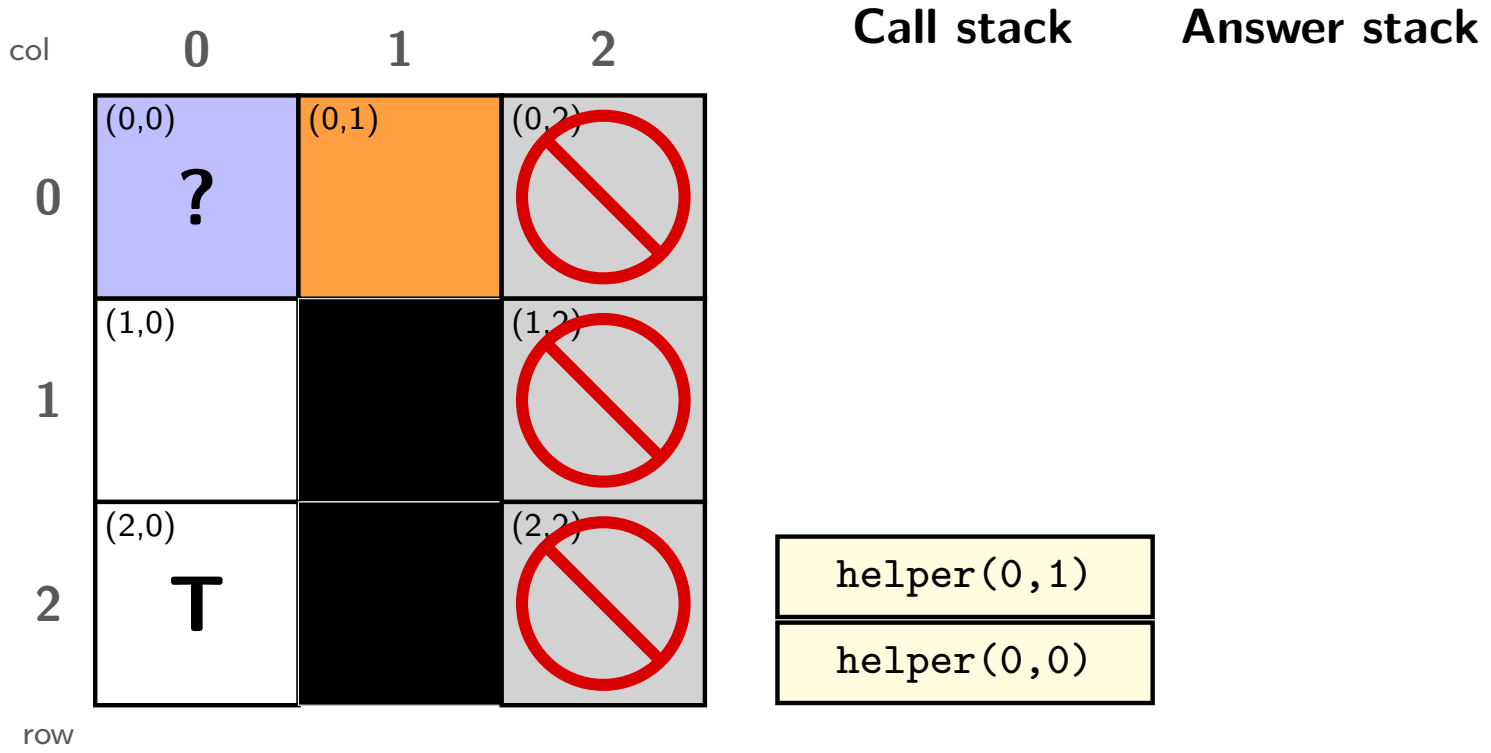
Back in `helper(0,2)`: the call to $(1, 2)$ returned nothing, and no other unvisited open neighbor remains.

Slip Question 3, Step 10 of 18



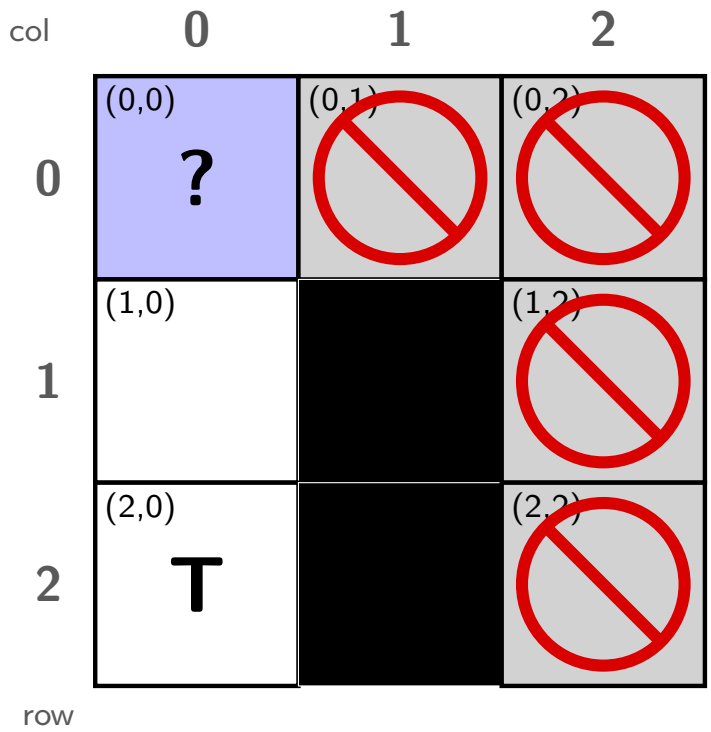
$(0, 2)$ **returns empty-handed**: ? becomes a stop sign. Backtrack to $(0, 1)$.

Slip Question 3, Step 11 of 18



Back in `helper(0,1)`: its only choice, $(0, 2)$, failed, and no other unvisited open neighbor remains.

Slip Question 3, Step 12 of 18

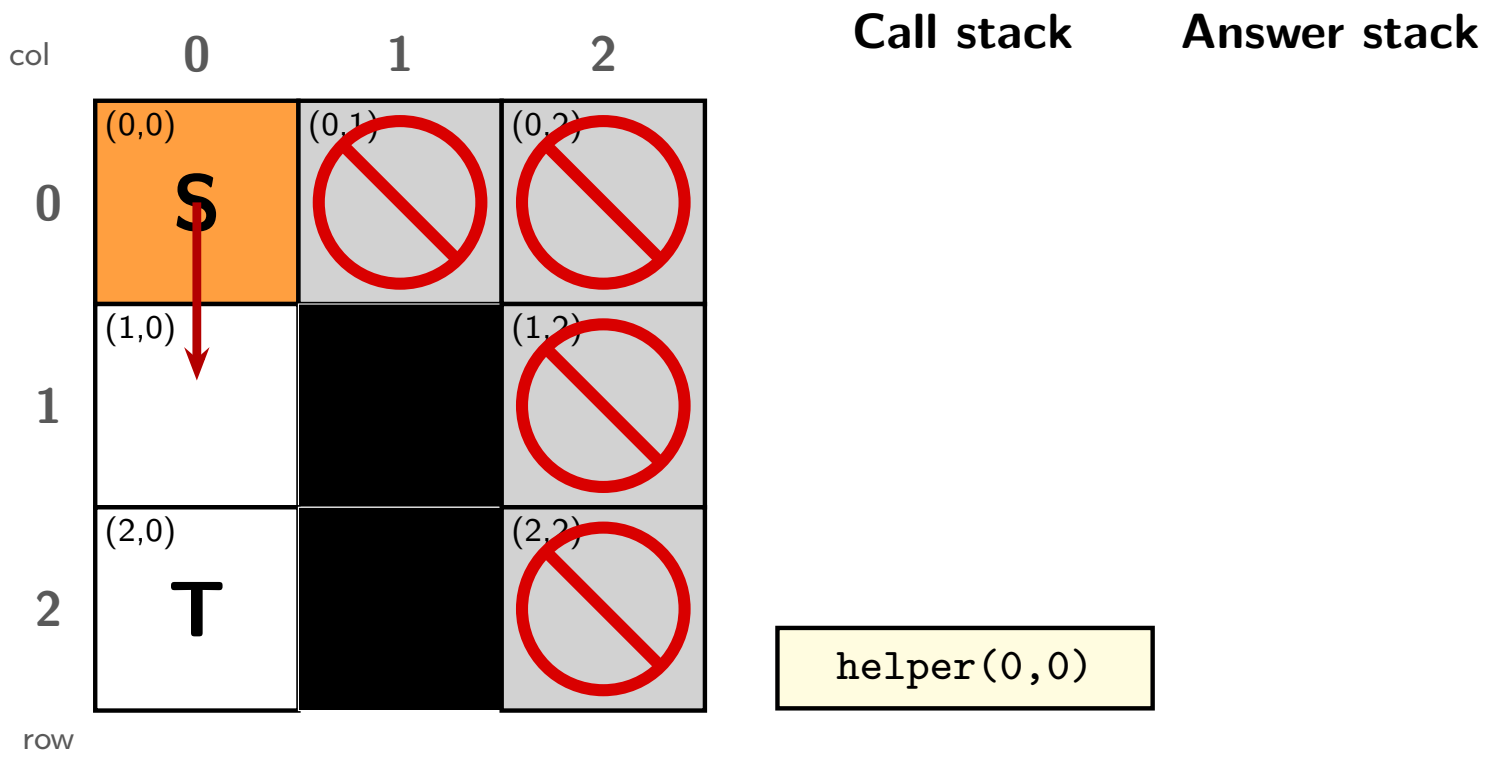


Call stack **Answer stack**

`helper(0,0)`

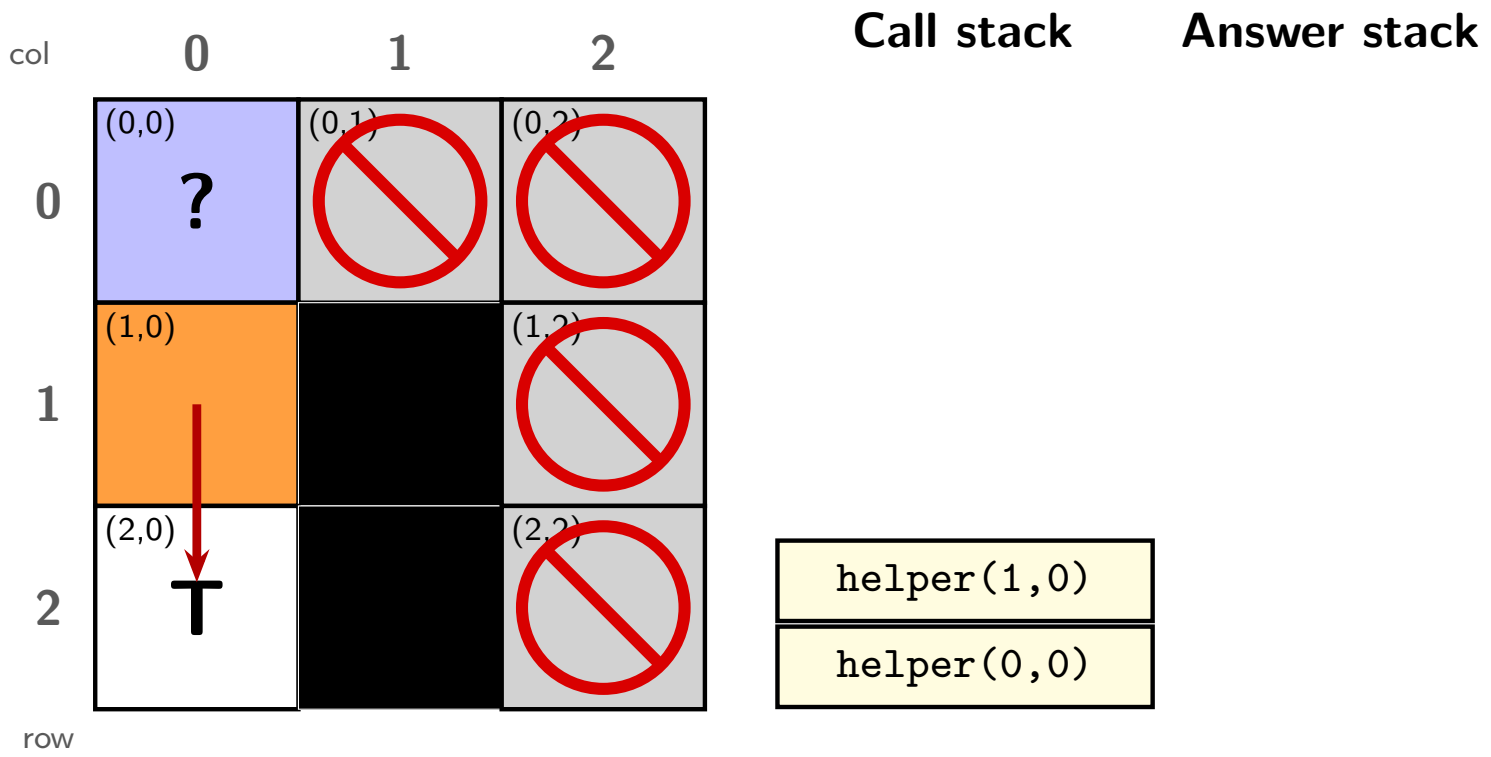
$(0, 1)$ **returns empty-handed**: ? becomes a stop sign. Backtrack to $(0, 0)$. Those four cells stay marked visited, so we never explore them again.

Slip Question 3, Step 13 of 18



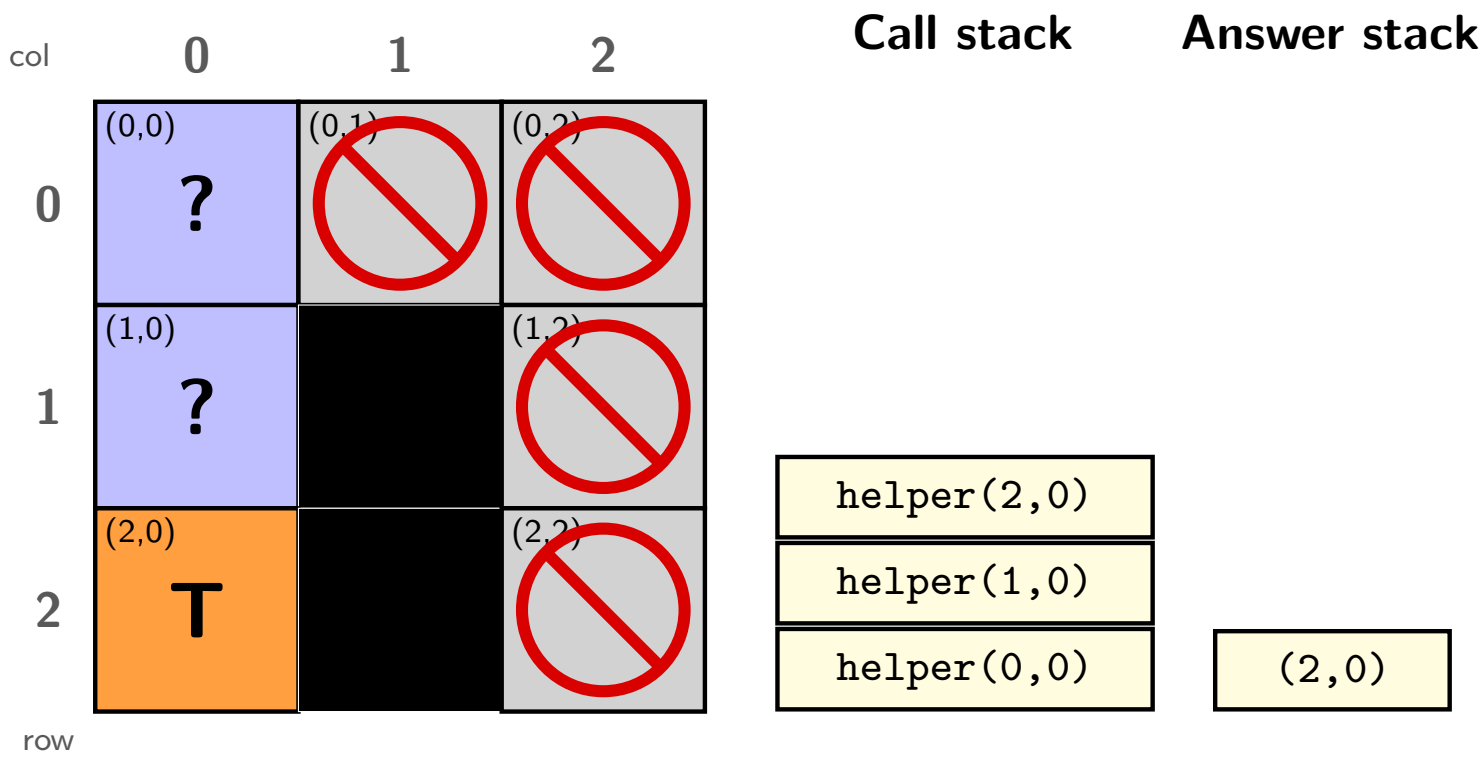
Back in `helper(0,0)`: the **right** neighbor failed. Next in the order: up is outside the grid, so try **down**: $(1, 0)$ is open and unvisited. **Call** `helper(1,0)`.

Slip Question 3, Step 14 of 18



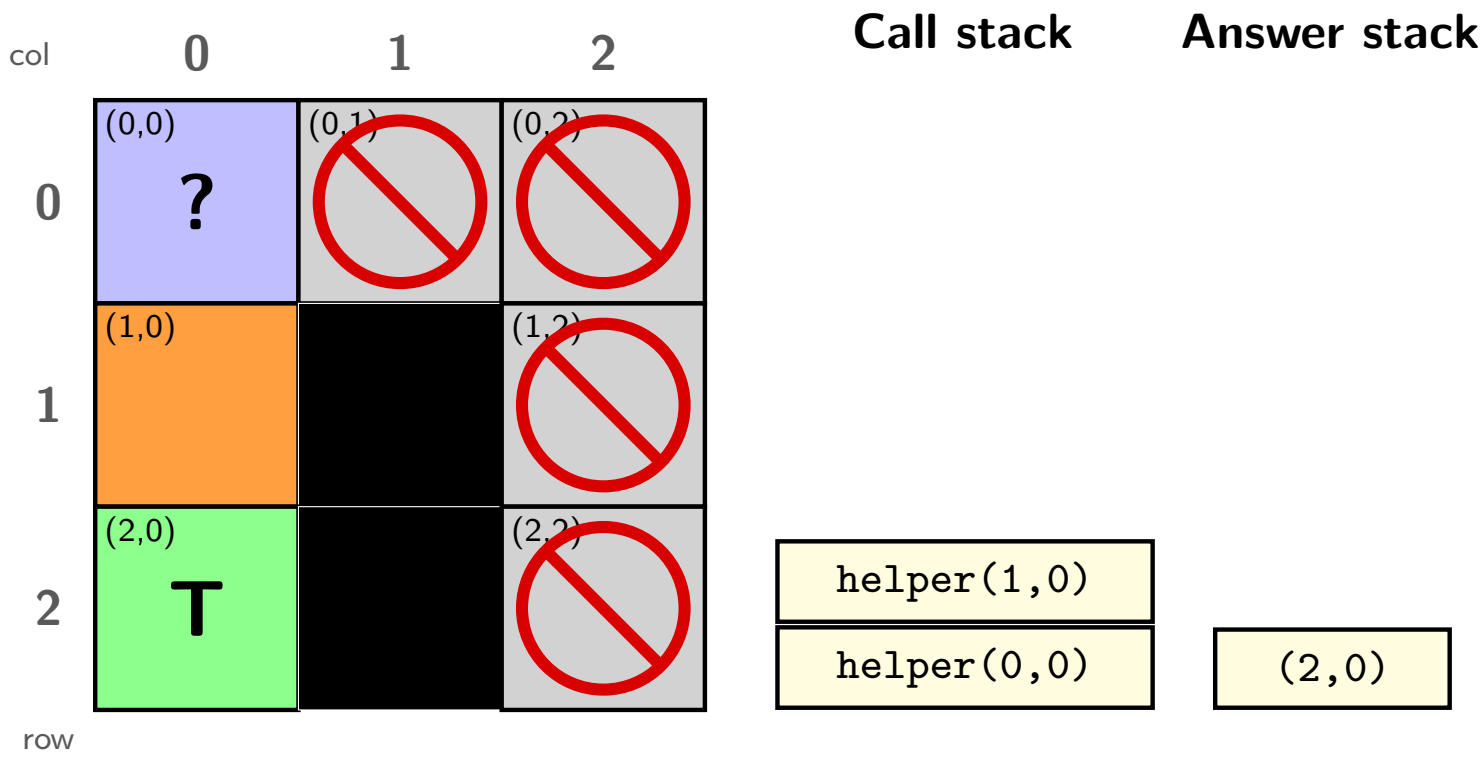
`helper(1,0)`: left is outside, right is a wall, up is visited. Down, $(2, 0)$, is open and unvisited. **Call** `helper(2,0)`.

Slip Question 3, Step 15 of 18



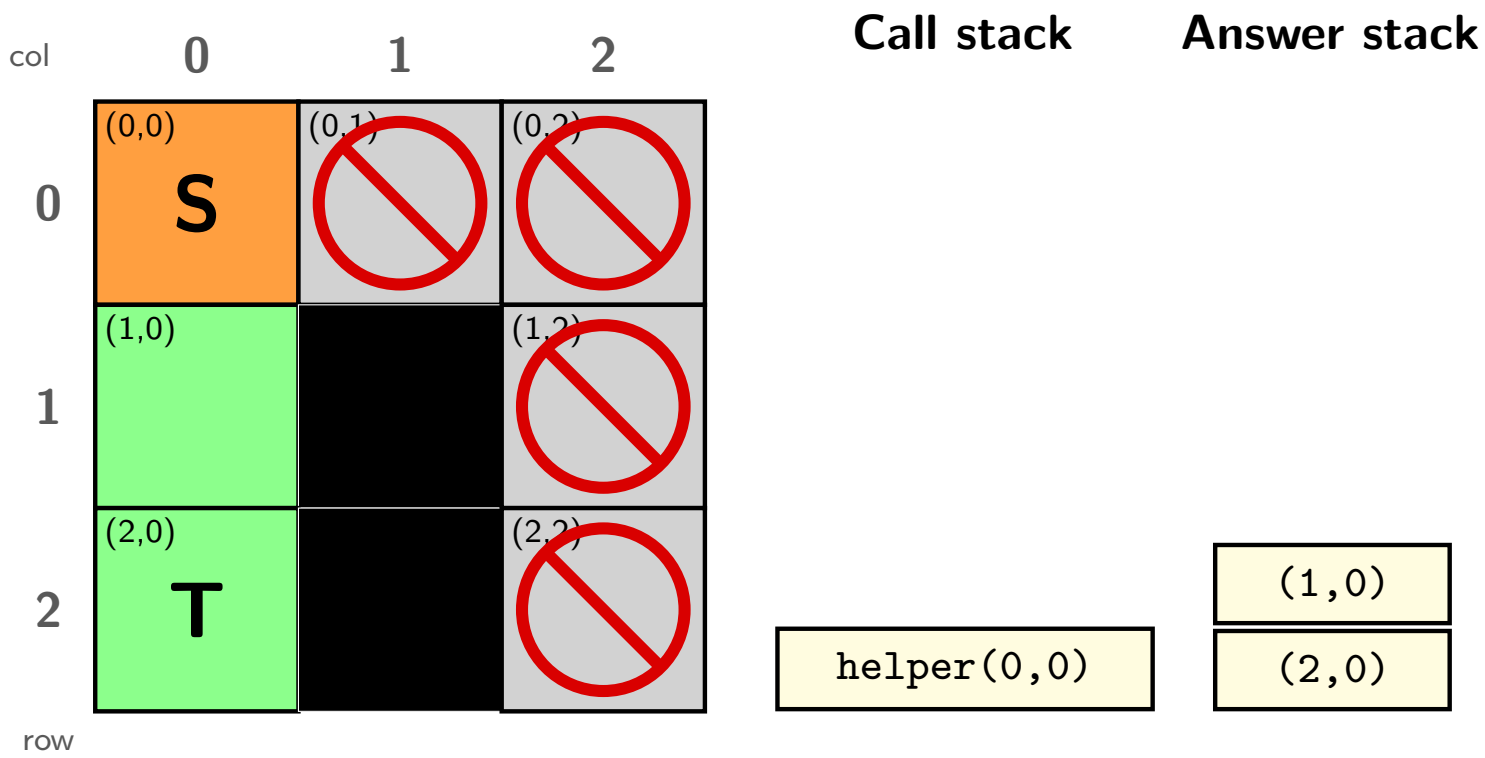
`helper(2,0)`: this cell is the **target**. **Success**: push $(2, 0)$ onto the answer stack. This is the first time the answer stack is not empty. Then `helper(2,0)` returns to `helper(1,0)`; its frame is still on the call stack until that return happens.

Slip Question 3, Step 16 of 18



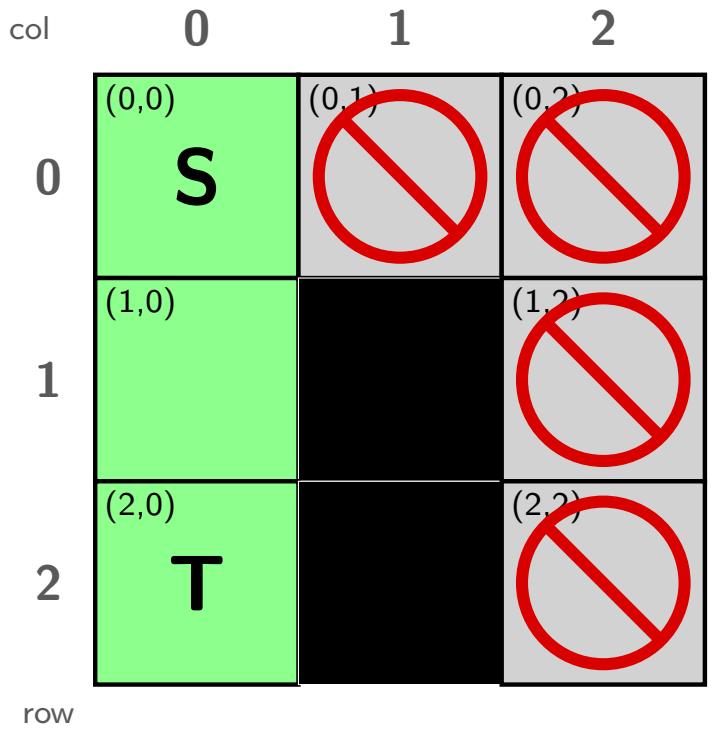
Back in `helper(1,0)`. The answer stack is **not empty**, so the call below us succeeded: push $(1, 0)$ and return immediately (no more neighbors are tried).

Slip Question 3, Step 17 of 18

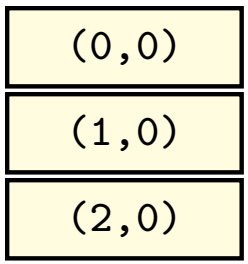


Back in `helper(0,0)`. The answer stack is not empty, so push $(0, 0)$ and return.

Slip Question 3, Step 18 of 18



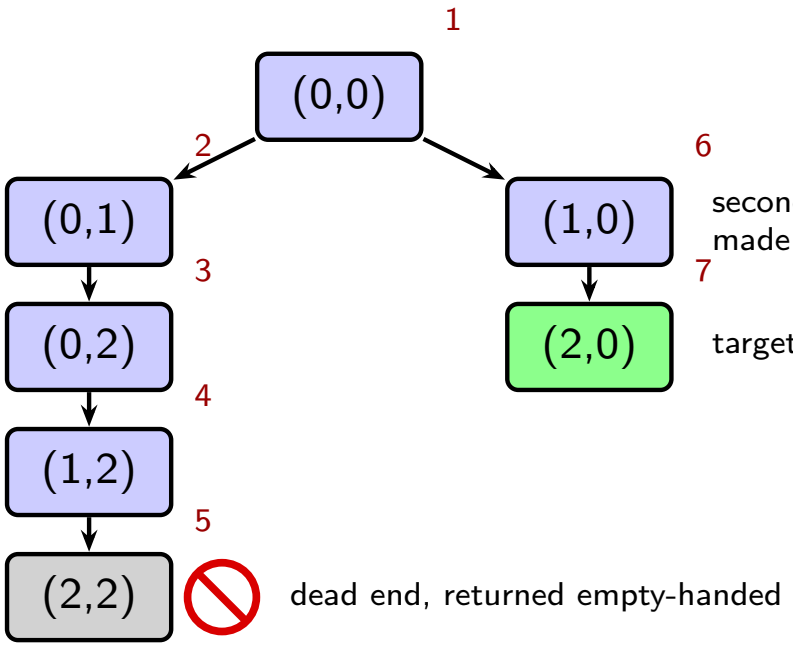
Call stack **Answer stack**



Done. The answer stack holds the route with the start on top: $(0,0) \to (1,0) \to (2,0)$. The stop signs mark the dead-end detour that was tried and abandoned. **Backtracking** was just a failed call returning, and **success** is the answer stack being filled while the calls return.

The Same Search as a Recursive-Call Tree

Each node is one call, `helper(maze, curr, target, visited, pathStack)`; the label is its `curr`.

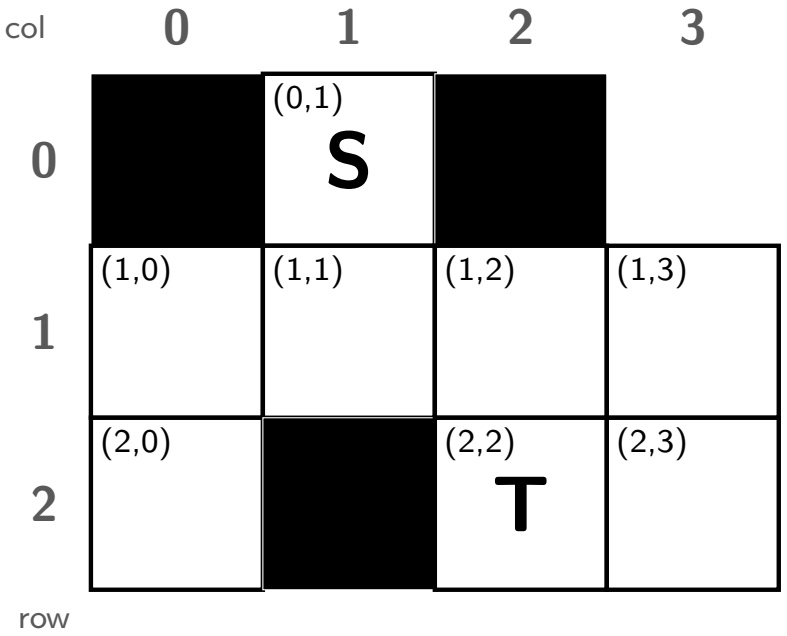


Final route (only the successful branch):

(0,0)→(1,0)→(2,0)

Small numbers: order of the calls. This tree is neither the maze nor the answer. The maze says who the neighbors are; the tree says which calls happened and which call made each recursive call; the route is only the successful branch.

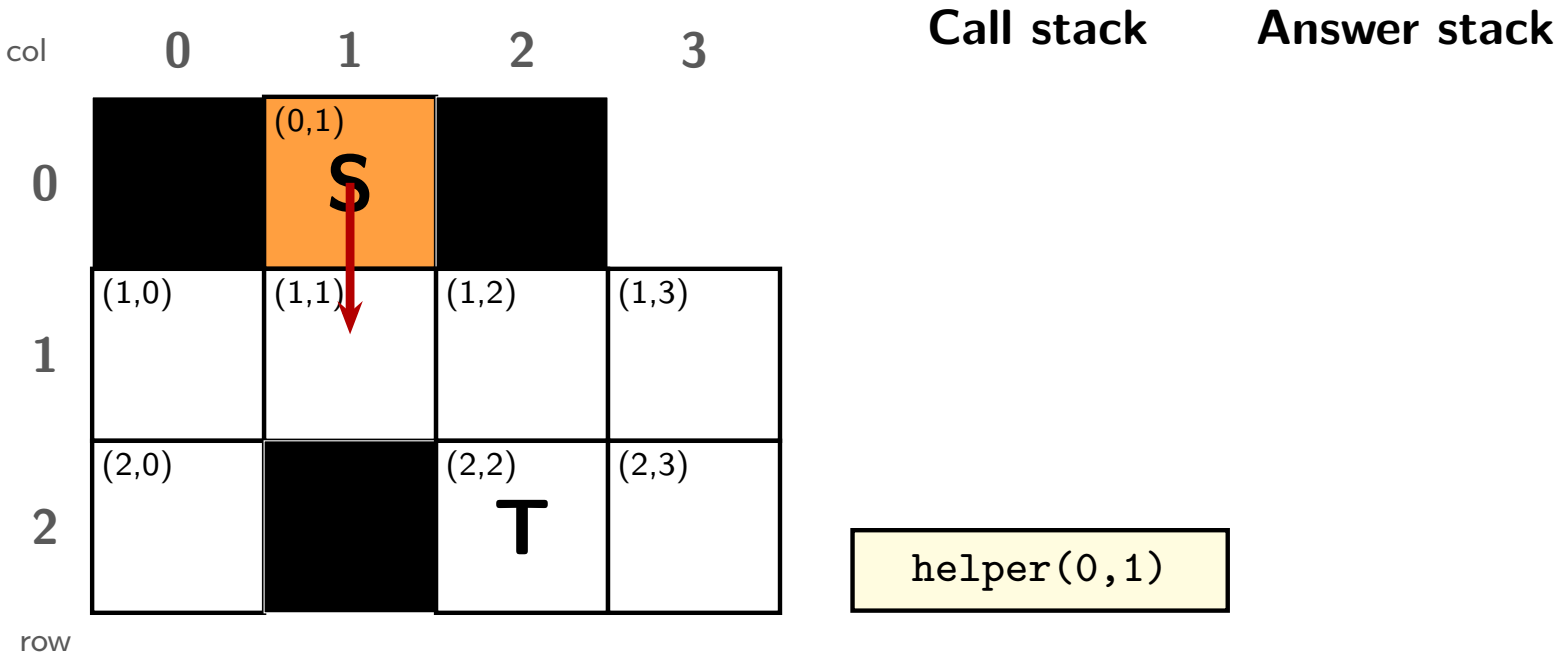
Example 2: Two Choices, and a Shorter Route



row 0 has only 3 cells:
$(0, 3)$ does not exist

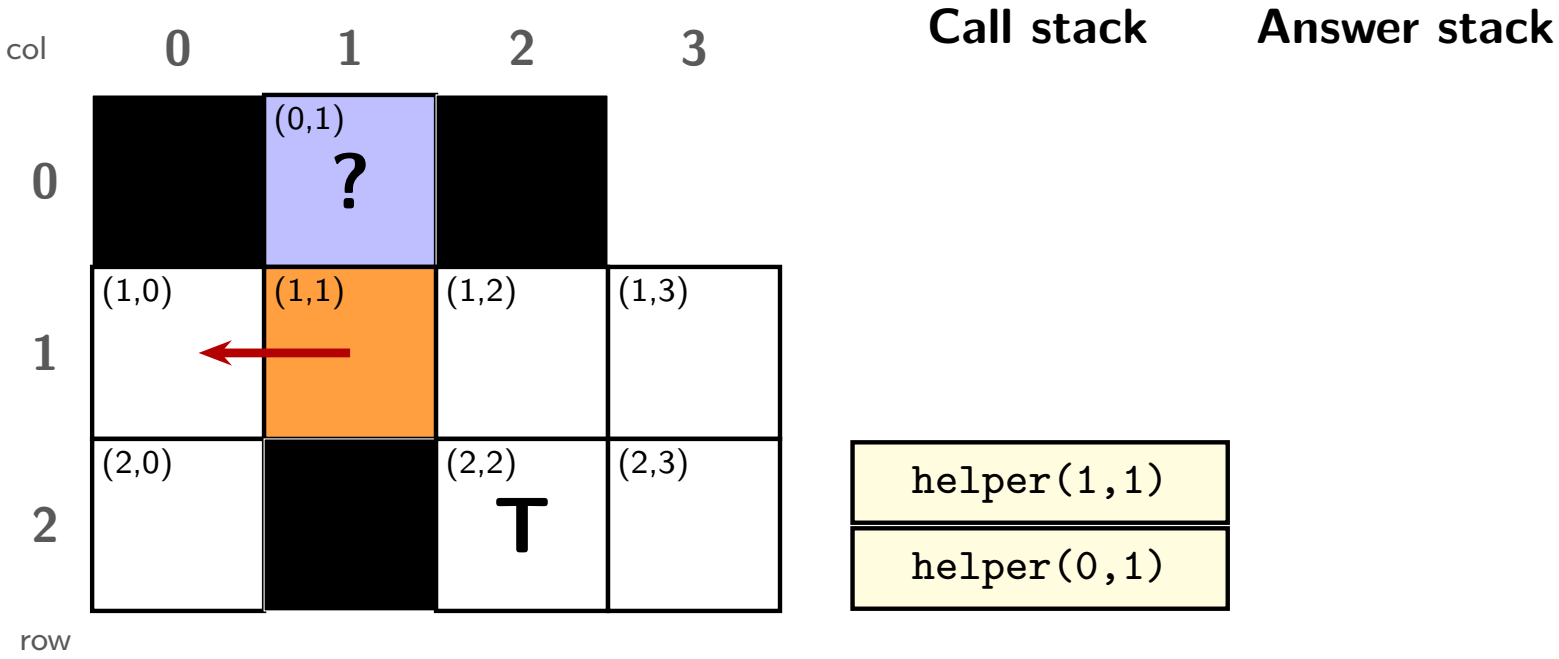
A **ragged** maze. Start $(0, 1)$, target $(2, 2)$, same rule: left, right, up, down. Two things to watch: junction $(1, 1)$ has two ways to go, and there is an obvious short route $(0, 1) \to (1, 1) \to (1, 2) \to (2, 2)$. Will DFS find it?

Example 2, Step 1 of 18



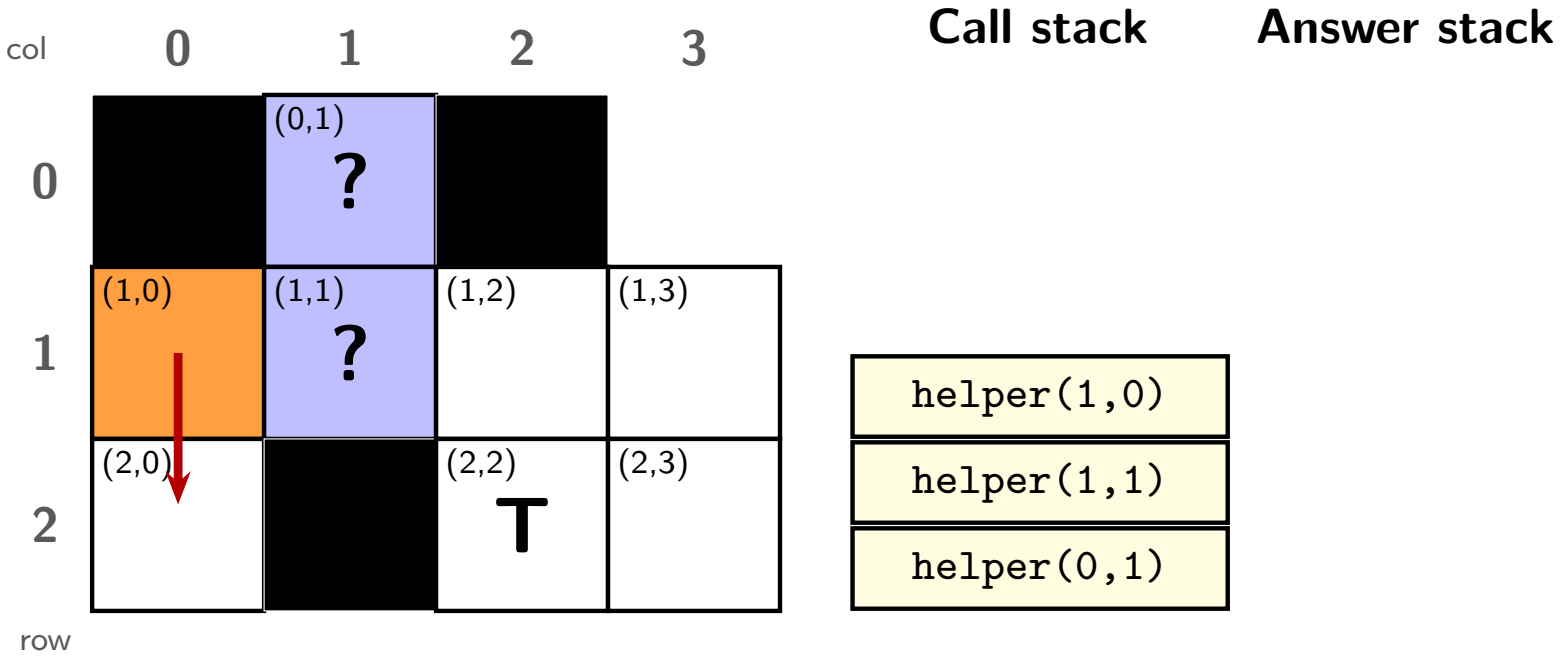
`helper(0,1)` (the start): mark it visited. Left and right are walls, up is outside the grid. Down, $(1, 1)$, is open: **call** `helper(1,1)`.

Example 2, Step 2 of 18



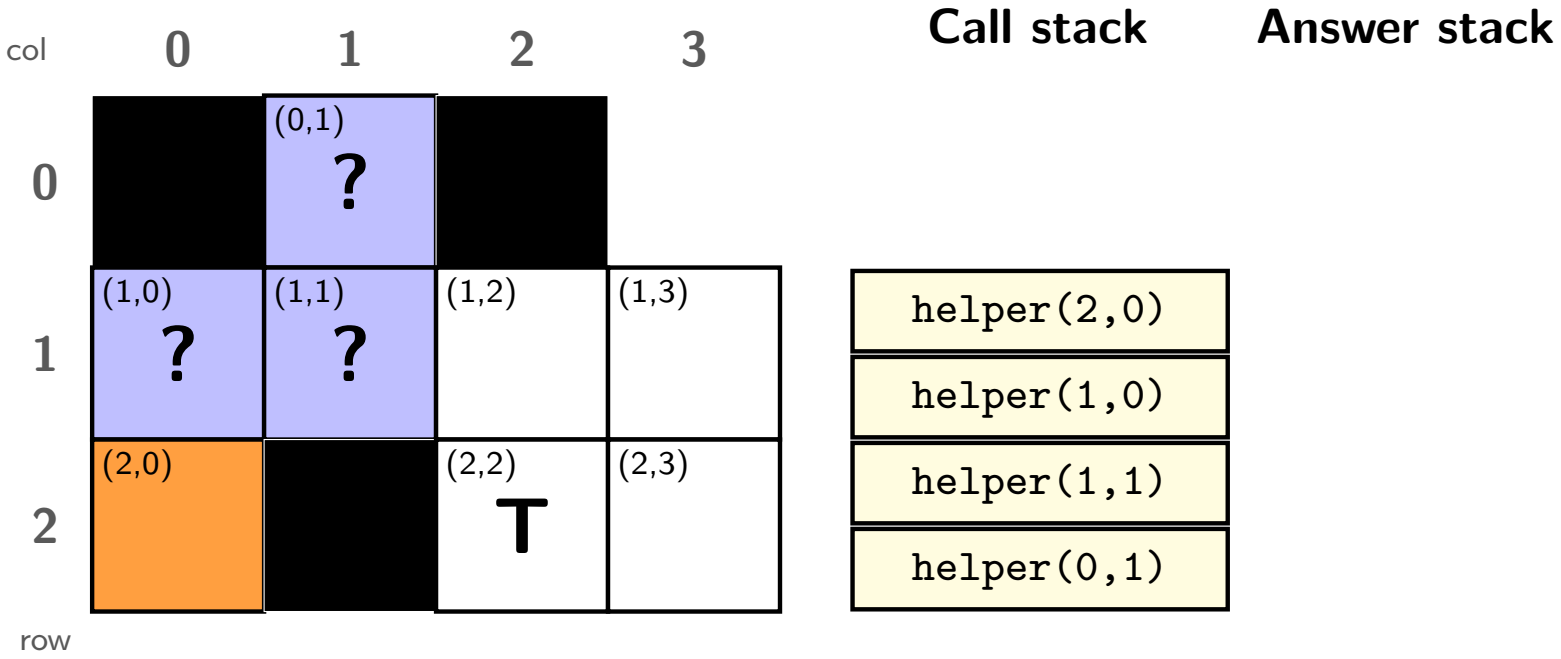
`helper(1,1)` is a junction with **choices**. The order is left, right, up, down, so try **left** first: $(1, 0)$ is open and unvisited. **Call** `helper(1,0)`.

Example 2, Step 3 of 18



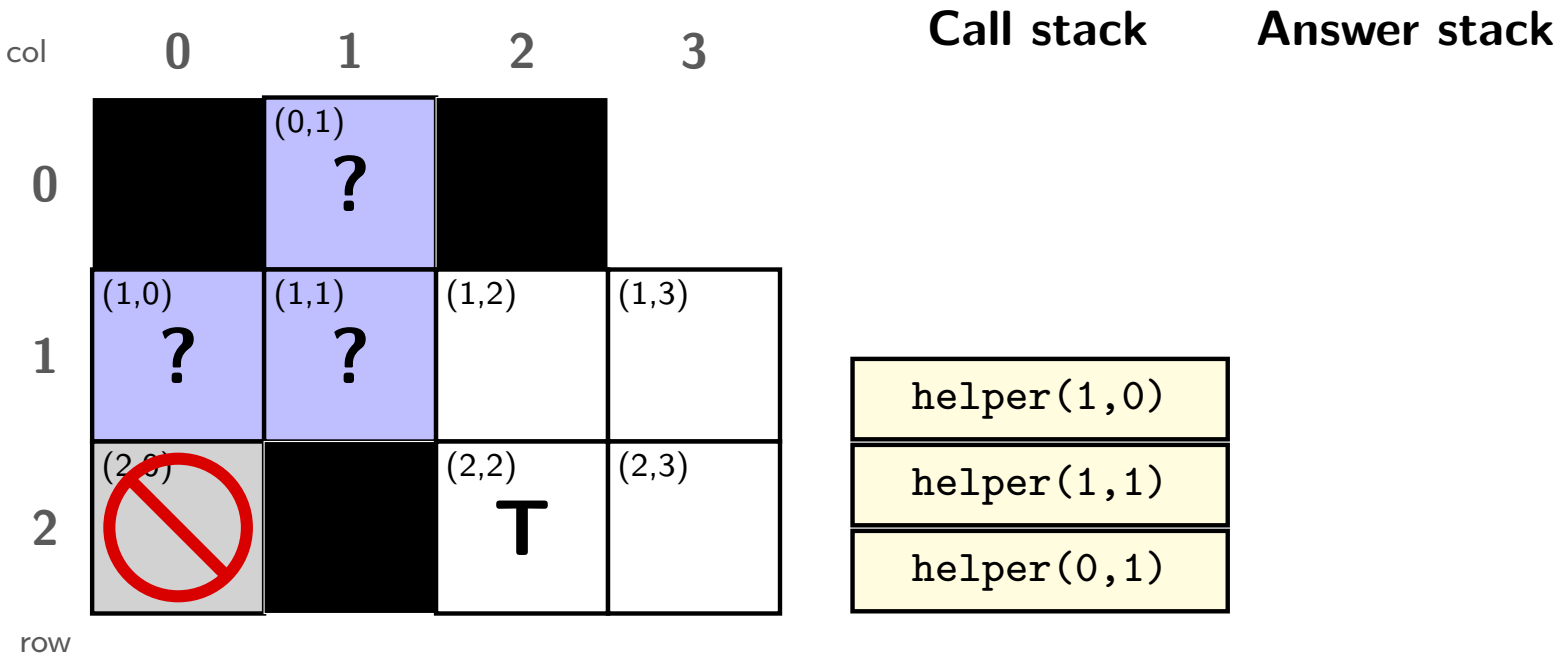
`helper(1,0)`: left is outside, right is visited, up is a wall. Only down, $(2, 0)$, is open. **Call** `helper(2,0)`.

Example 2, Step 4 of 18



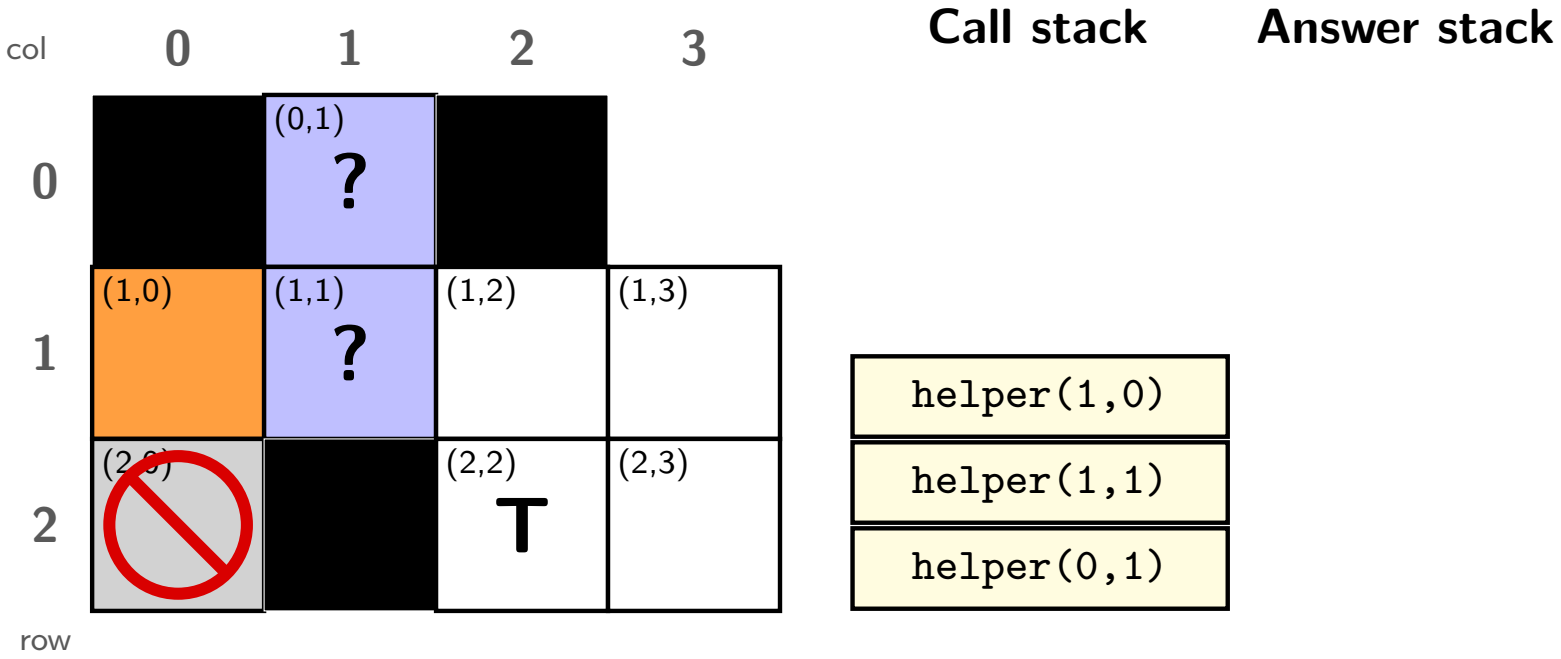
`helper(2,0)`: left and down are outside, right is a wall, up is visited.
A **dead end**: nothing to put on the answer stack.

Example 2, Step 5 of 18



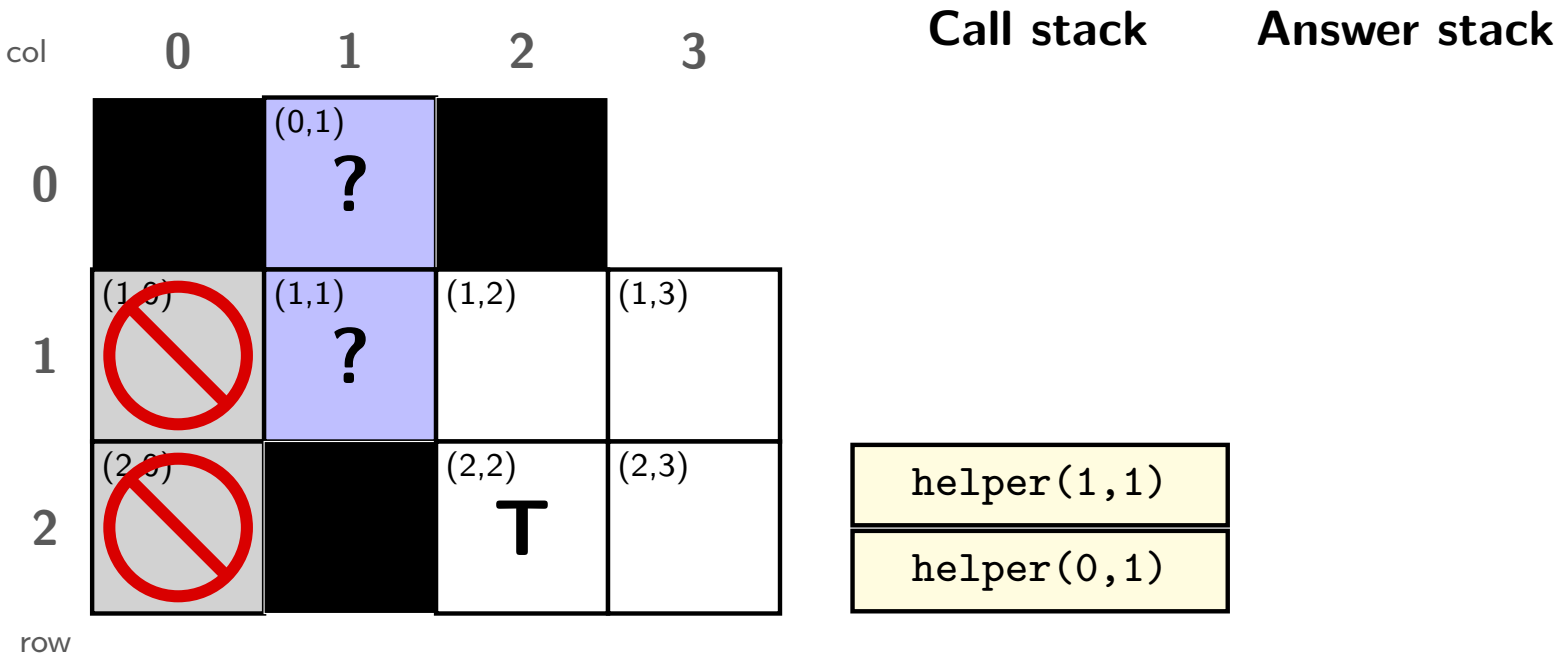
`helper(2,0)` **returns empty-handed**: stop sign. Backtrack to $(1, 0)$.

Example 2, Step 6 of 18



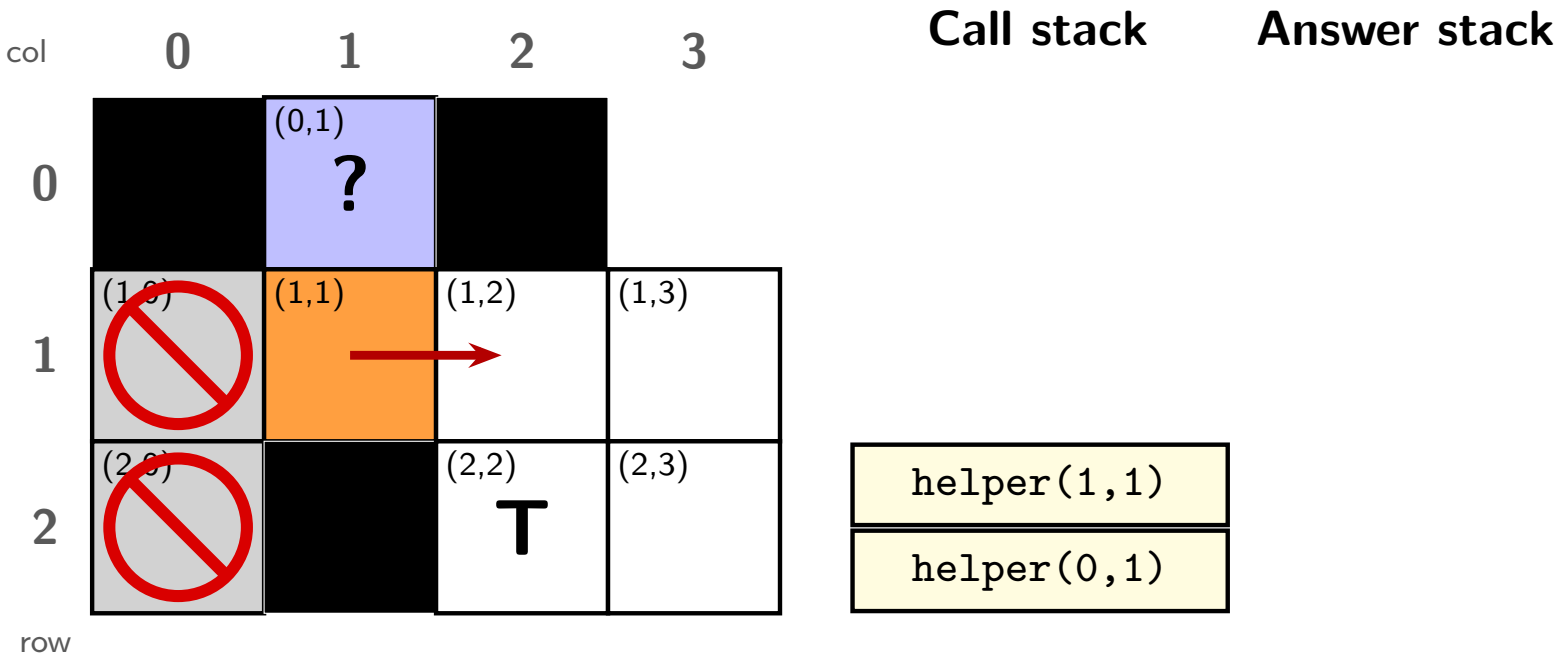
Back in `helper(1,0)`: the call returned nothing, and $(1, 0)$ has no other unvisited open neighbor.

Example 2, Step 7 of 18



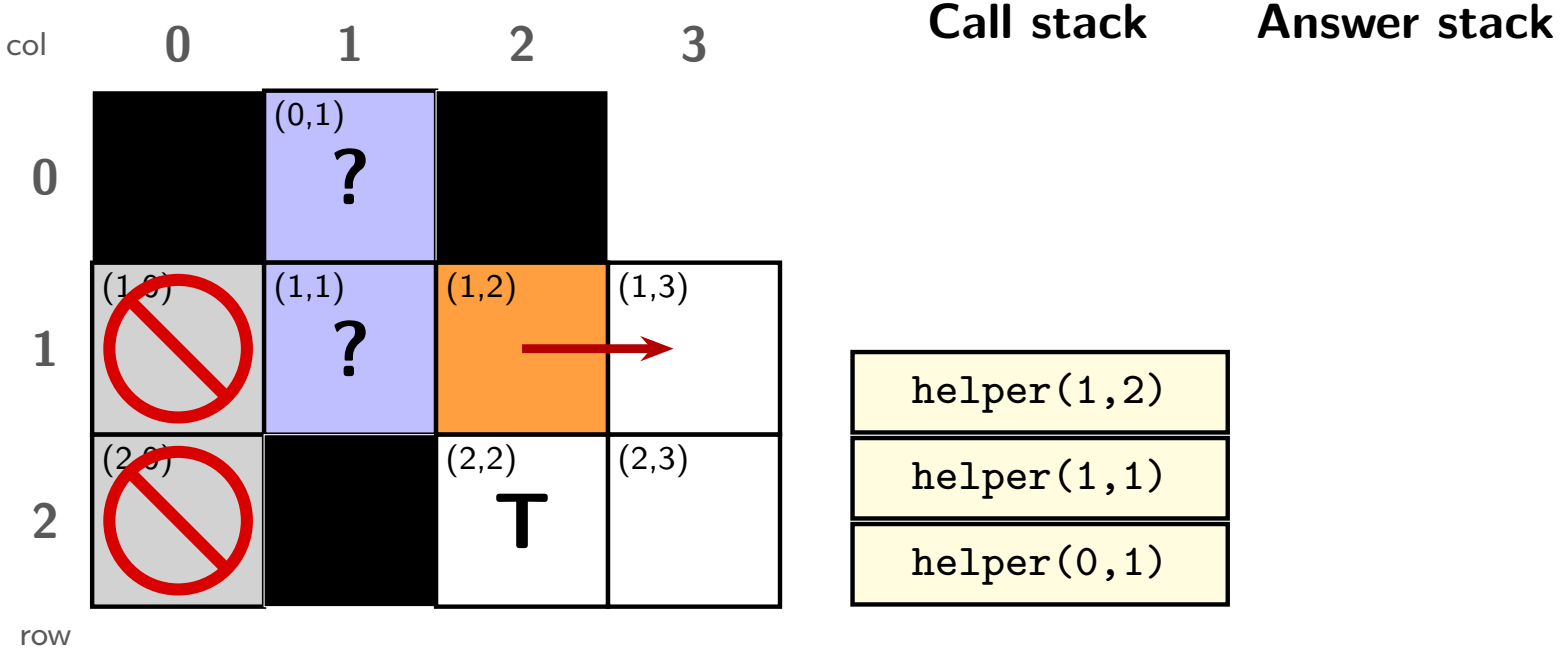
$(1, 0)$ **returns empty-handed**: stop sign. Backtrack to $(1, 1)$.

Example 2, Step 8 of 18



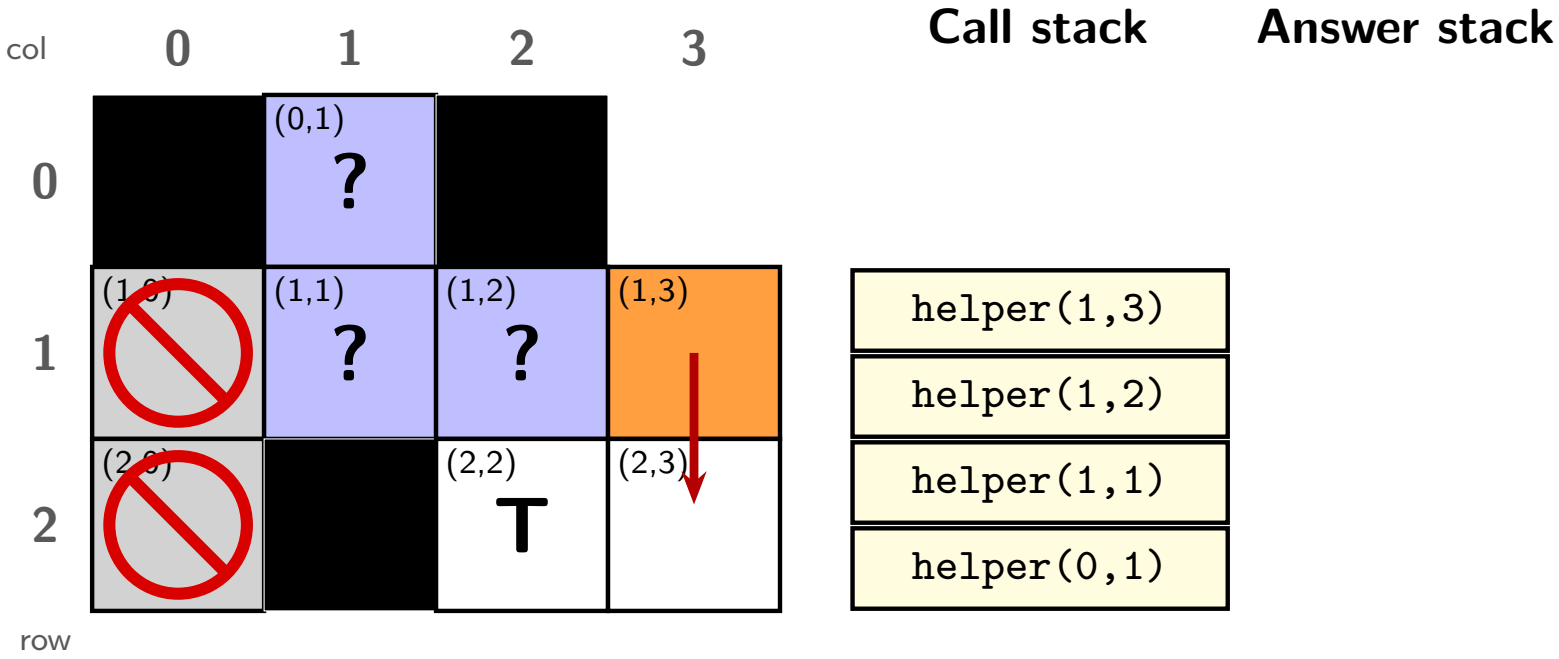
Back in `helper(1,1)`. The **left** choice failed, but this cell still has neighbors to try. **This is the new idea:** a failed call does not end the search at $(1, 1)$; it only moves on to the next neighbor in the order. Next is **right**: $(1, 2)$ is open and unvisited. **Call** `helper(1,2)`.

Example 2, Step 9 of 18



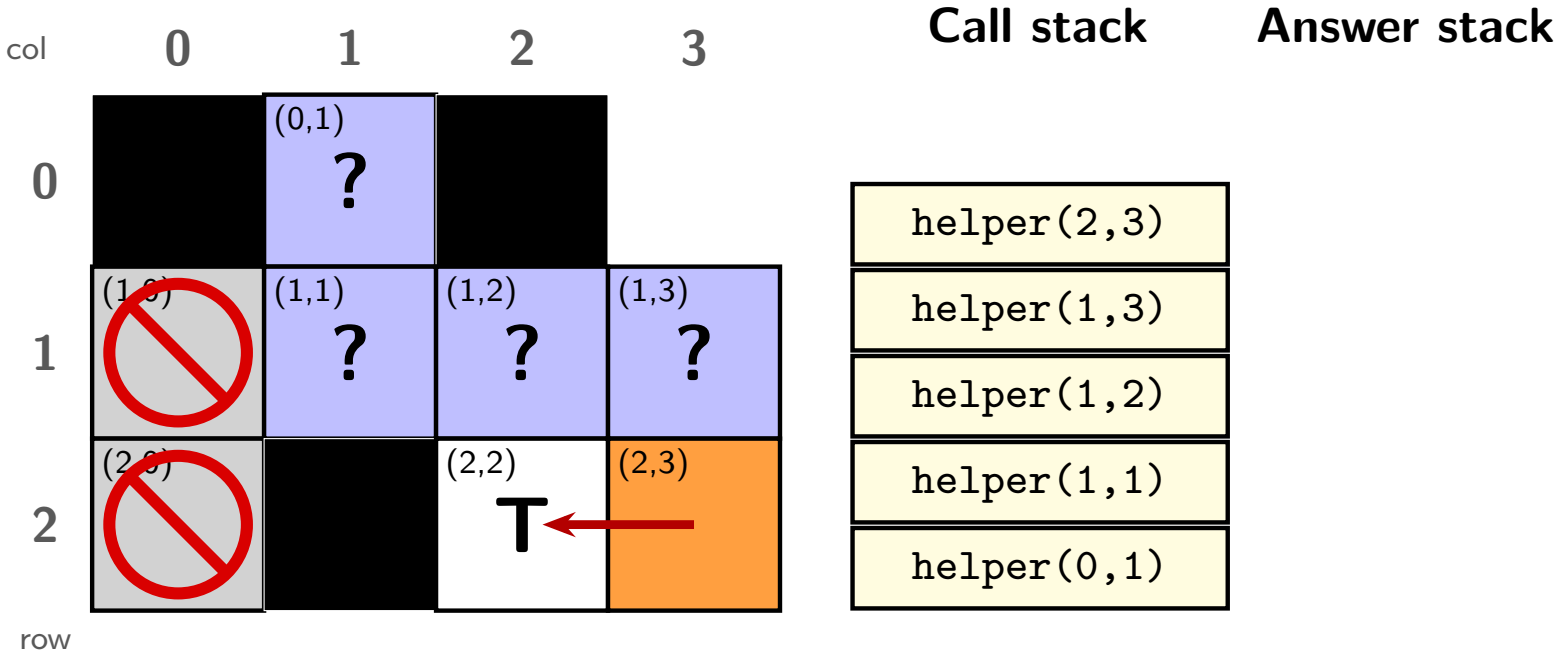
`helper(1,2)`: left is visited. Next in the order is **right**: $(1, 3)$ is open. **Call** `helper(1,3)`. Notice that the target $(2, 2)$ is directly **below** us, but **down is tried last**, so DFS goes right first.

Example 2, Step 10 of 18



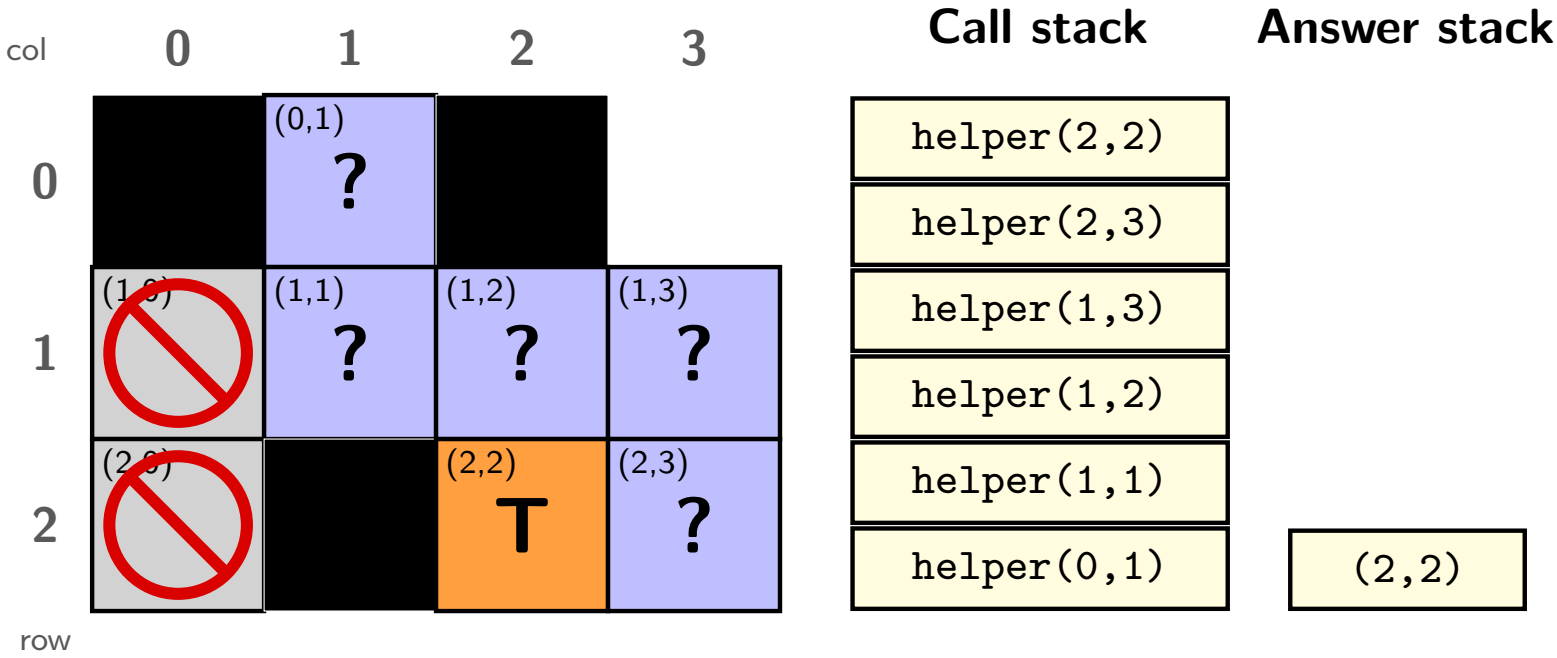
`helper(1,3)`: left is visited, right and up are outside the grid. Down, $(2, 3)$, is open. **Call** `helper(2,3)`.

Example 2, Step 11 of 18



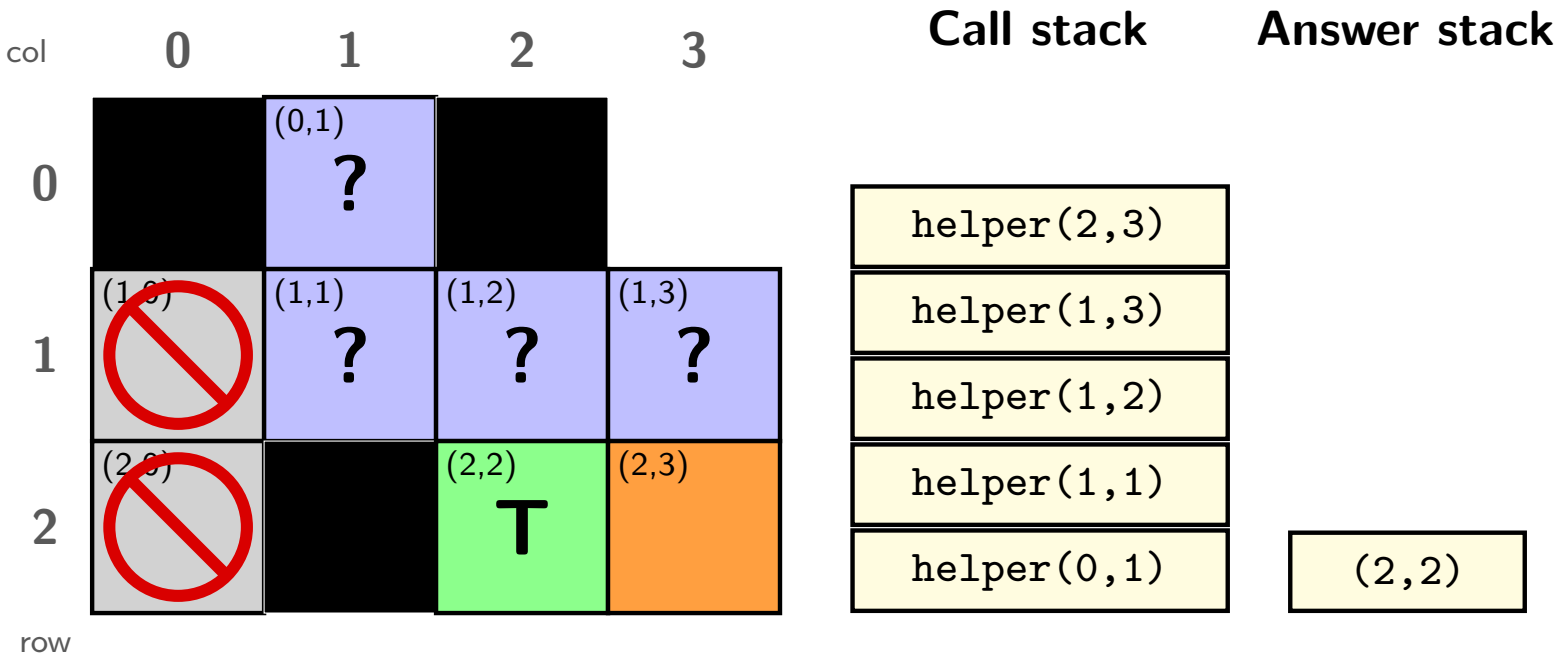
`helper(2,3)`: the left neighbor $(2, 2)$ is open and unvisited. **Call** `helper(2,2)`.

Example 2, Step 12 of 18



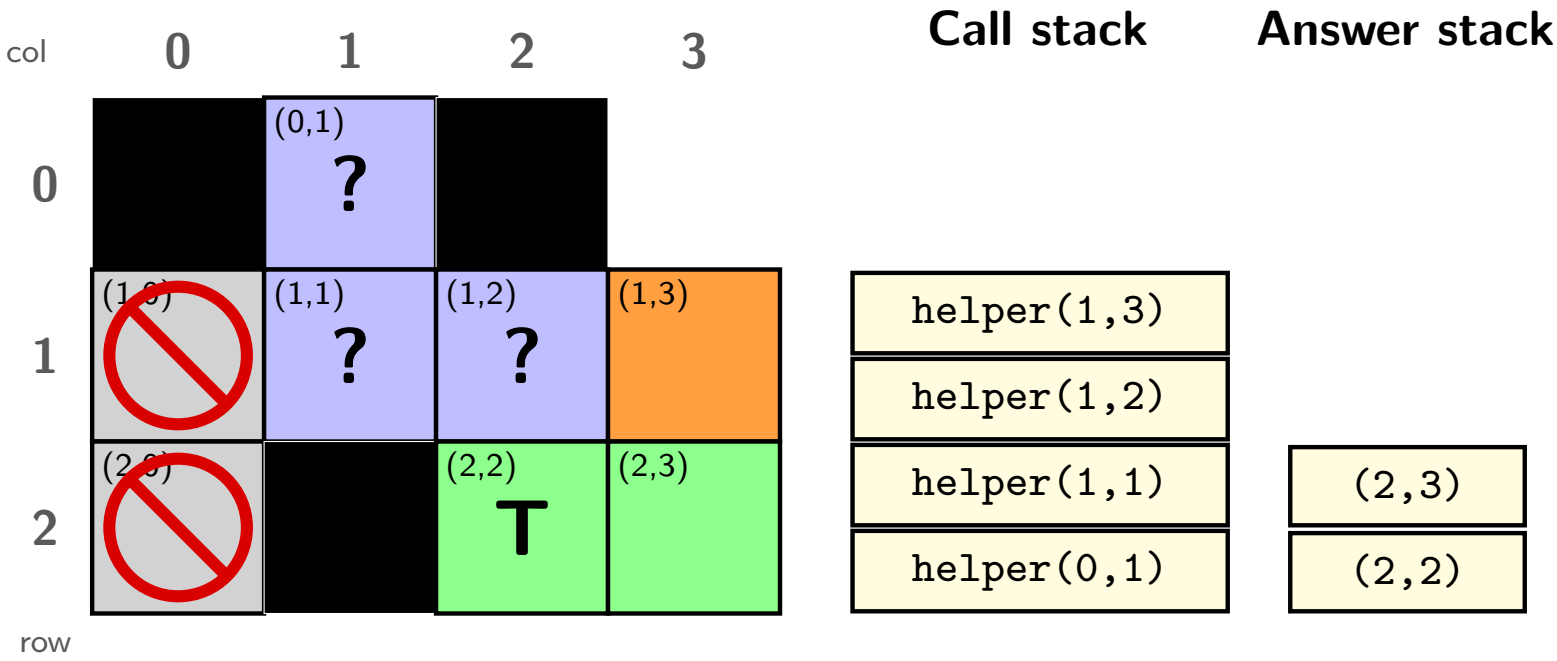
`helper(2,2)`: the **target**. Push $(2, 2)$ onto the answer stack. This is the first time the answer stack is not empty. Then it returns to `helper(2,3)`.

Example 2, Step 13 of 18



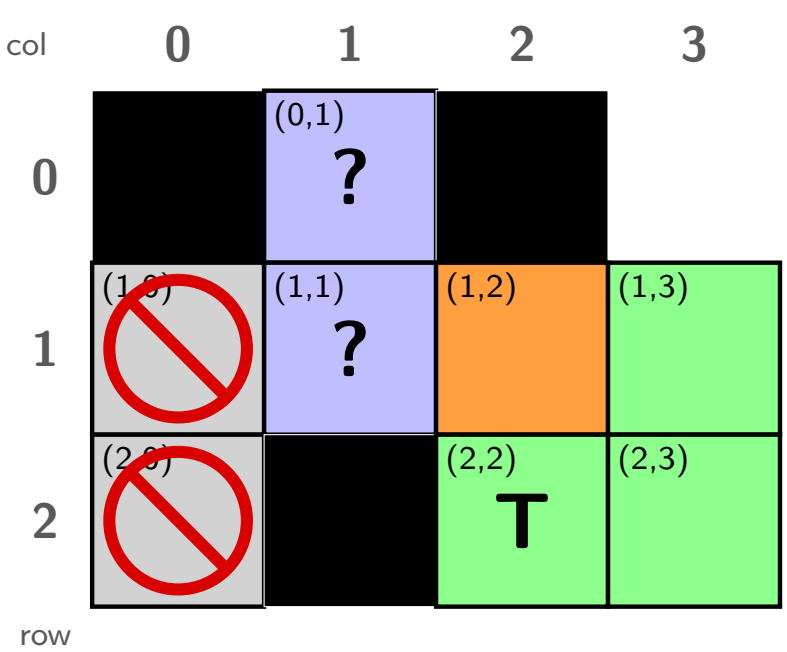
Back in `helper(2,3)`: the answer stack is **not empty**, so push $(2, 3)$ and return.

Example 2, Step 14 of 18



Back in `helper(1,3)`: not empty, so push $(1, 3)$ and return.

Example 2, Step 15 of 18



Call stack

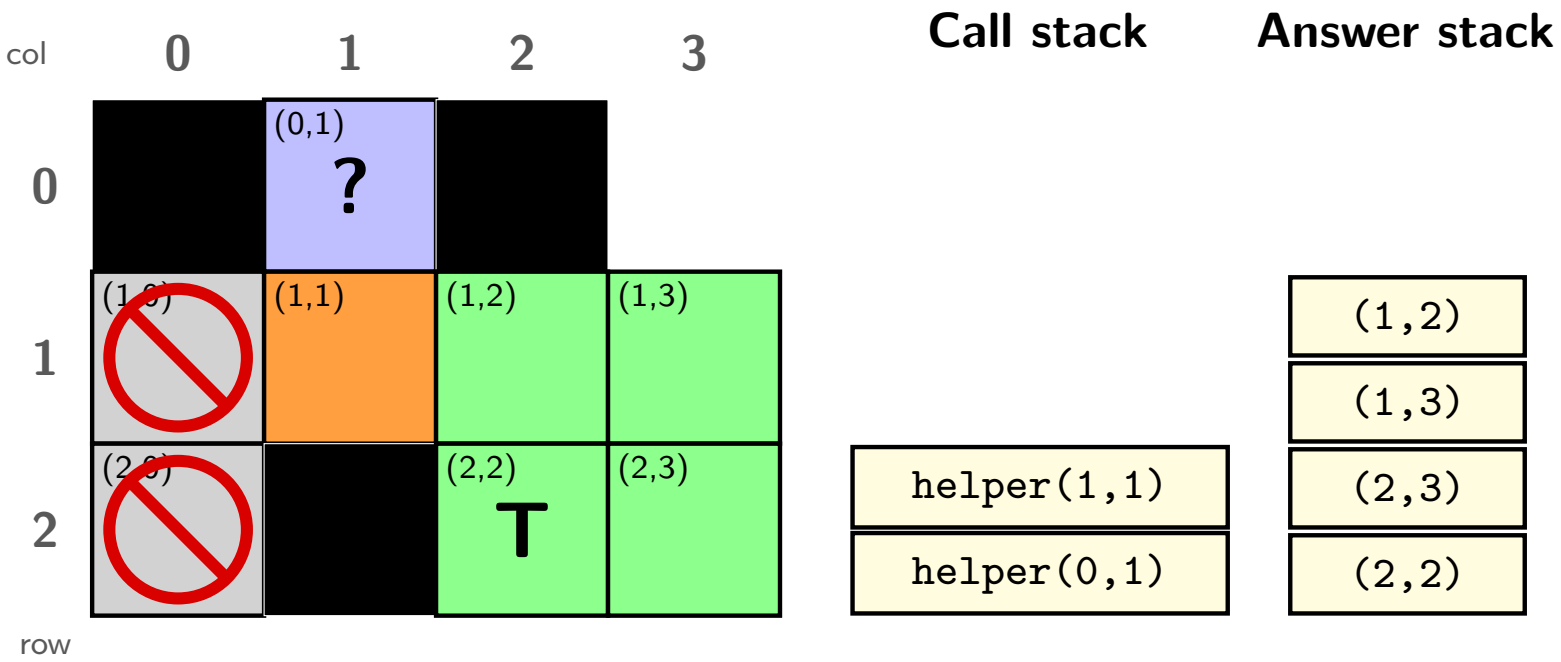
```
helper(1,2)
helper(1,1)
helper(0,1)
```

Answer stack

```
(1,3)
(2,3)
(2,2)
```

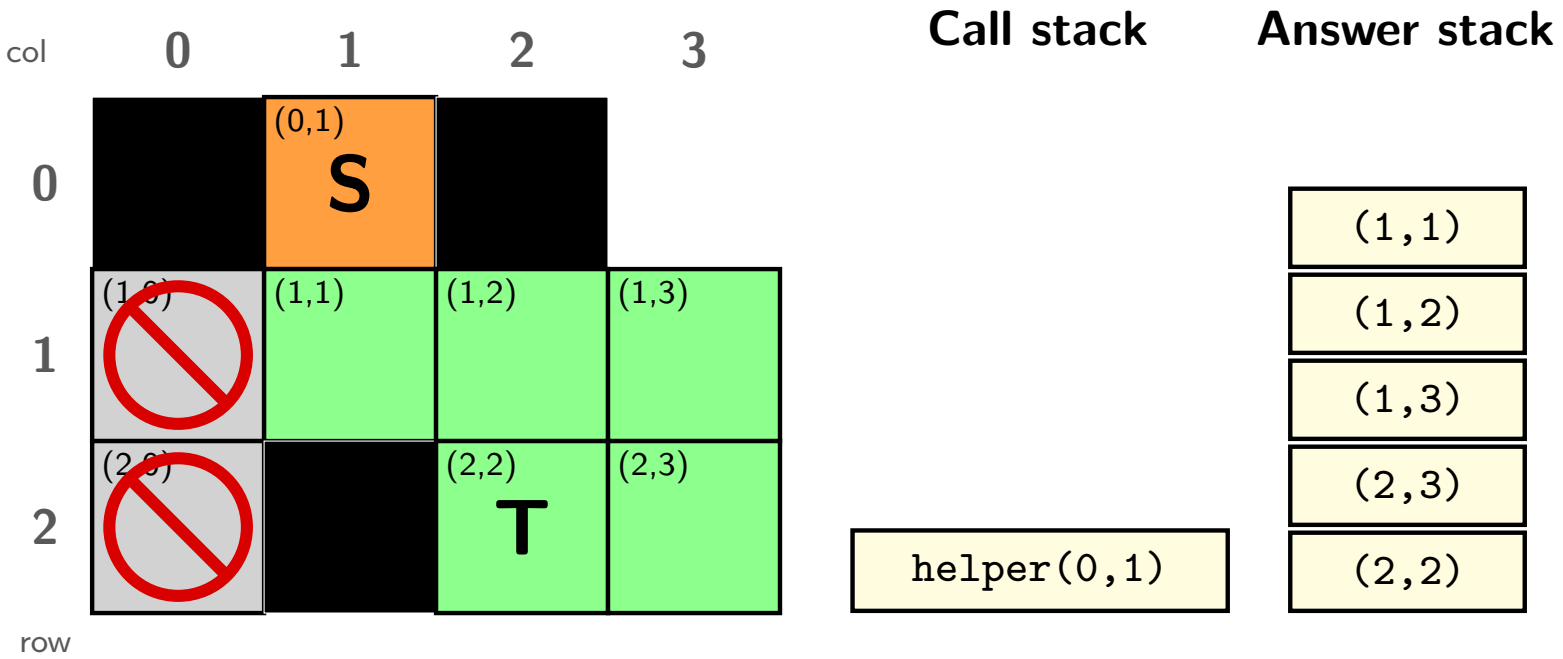
Back in `helper(1,2)`: not empty, so push $(1, 2)$ and return *immediately*. Its down neighbor $(2, 2)$ is never tried from here, although it is the target.

Example 2, Step 16 of 18



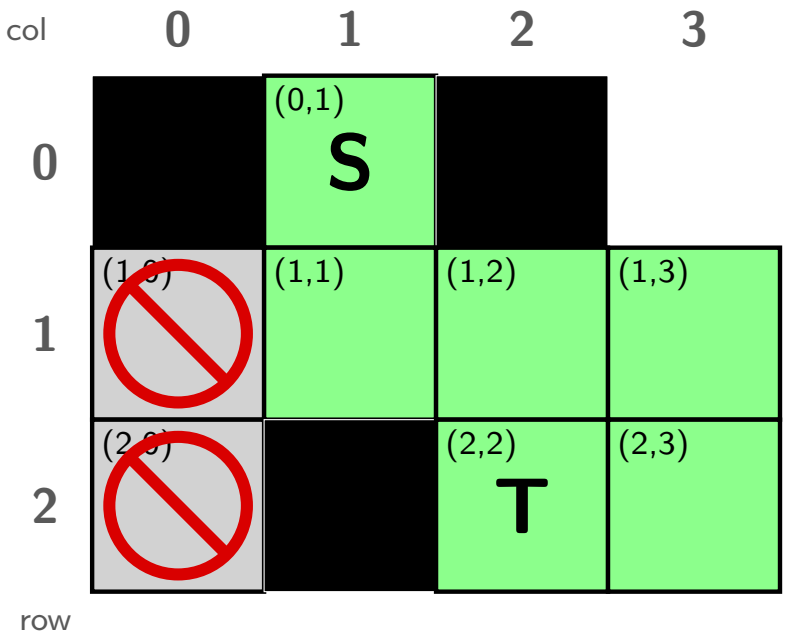
Back in `helper(1,1)`: the right choice succeeded. Push $(1, 1)$ and return immediately. The failed left branch stays marked visited and is not part of the route.

Example 2, Step 17 of 18



Back in `helper(0,1)`: the answer stack is not empty, so push $(0, 1)$ and return.

Example 2, Step 18 of 18



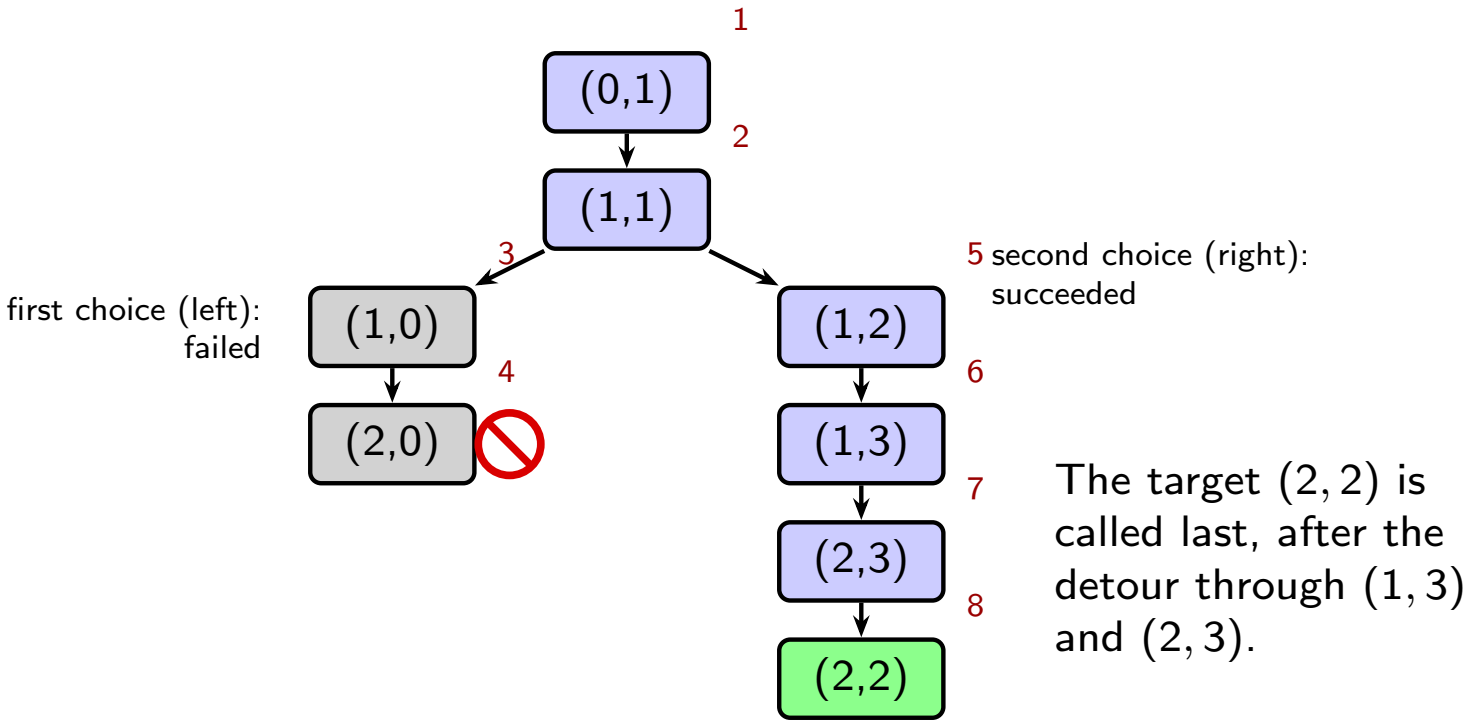
Call stack

Answer stack

(0,1)
(1,1)
(1,2)
(1,3)
(2,3)
(2,2)

Done. DFS returns the 6-cell route $(0,1) \to (1,1) \to (1,2) \to (1,3) \to (2,3) \to (2,2)$. A **shorter** 4-cell route exists. Does DFS promise the shortest route? **No.**

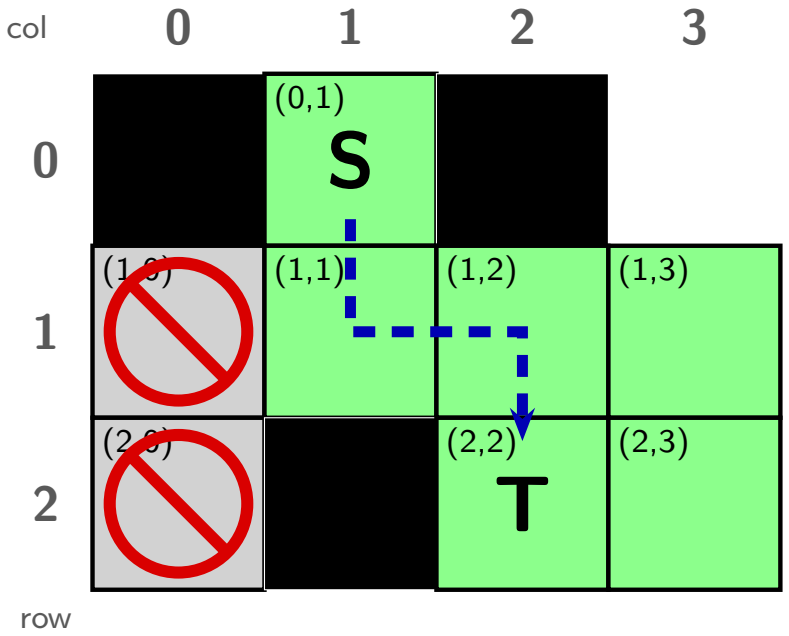
Example 2 as a Recursive-Call Tree



The target $(2, 2)$ is called last, after the detour through $(1, 3)$ and $(2, 3)$.

The tree is a single chain after $(1, 1)$'s second call: the route is every node on the successful branch, 6 cells. Down was tried after right at $(1, 2)$, so the 1-step shortcut $(1, 2) \to (2, 2)$ was never taken.

DFS Finds *A* Route, Not the Shortest



dashed blue: shortest route, 4 cells

green: the route DFS returned, 6 cells

gray + stop sign: explored, abandoned

DFS only promises a route *if one exists*. With the order left, right, up, down it went right at $(1, 2)$ before trying down, so it took the long way around. To get a shortest route you need **breadth-first search**; that is Task B.